# Xlib API overview

**Nathan Warner**


Northern Illinois
University

Computer Science
Northern Illinois University
United States

## Contents

# Headers

- **Headers**

```
<X11/Xlib.h>          // main Xlib declarations
<X11/X.h>             // X protocol types/constants
<X11/Xcms.h>          // Xcms color management
<X11/Xutil.h>         // ICCCM / utility functions
<X11/Xresource.h>     // resource manager
<X11/Xatom.h>         // predefined atoms
<X11/cursorfont.h>    // standard cursor font symbols
<X11/keysymdef.h>     // KeySym values
<X11/keysym.h>        // includes KeySym groups
<X11/Xlibint.h>       // extension/internal Xlib symbols
<X11/Xproto.h>        // basic X protocol symbols
<X11/Xprotostr.h>     // protocol structures
<X11/X10.h>           // X10 compatibility
```

## Types and constants

- **Core types**

```
Display
Screen
Window
Pixmap
Colormap
Cursor
Font
GContext
Drawable
Atom
KeyCode
KeySym
Bool
Status
XID
XPointer
```

- **Resource ID types**

```
Window
Font
Pixmap
Colormap
Cursor
GContext
```

- **Constants**:

```
True
False
None
NULL
```

## Atoms and prefixes

- **Atoms**:

```
0   XA_PRIMARY                  1
1   XA_SECONDARY                2
2   XA_ARC                      3
3   XA_ATOM                     4
4   XA_BITMAP                   5
5   XA_CARDINAL                 6
6   XA_COLORMAP                 7
7   XA_CURSOR                   8
8   XA_CUT_BUFFER0              9
9   XA_CUT_BUFFER1              10
10  XA_CUT_BUFFER2              11
11  XA_CUT_BUFFER3              12
12  XA_CUT_BUFFER4              13
13  XA_CUT_BUFFER5              14
14  XA_CUT_BUFFER6              15
15  XA_CUT_BUFFER7              16
16  XA_DRAWABLE                 17
17  XA_FONT                     18
18  XA_INTEGER                  19
```

```
0   XA_PIXMAP                   20
1   XA_POINT                    21
2   XA_RECTANGLE                22
3   XA_RESOURCE_MANAGER         23
4   XA_RGB_COLOR_MAP            24
5   XA_RGB_BEST_MAP             25
6   XA_RGB_BLUE_MAP             26
7   XA_RGB_DEFAULT_MAP          27
8   XA_RGB_GRAY_MAP             28
9   XA_RGB_GREEN_MAP            29
10  XA_RGB_RED_MAP              30
11  XA_STRING                   31
12  XA_VISUALID                 32
13  XA_WINDOW                   33
14  XA_WM_COMMAND               34
15  XA_WM_HINTS                 35
16  XA_WM_CLIENT_MACHINE        36
17  XA_WM_ICON_NAME             37
18  XA_WM_ICON_SIZE             38
```

```
XA_WM_NAME                39
XA_WM_NORMAL_HINTS        40
XA_WM_SIZE_HINTS          41
XA_WM_ZOOM_HINTS          42
XA_MIN_SPACE              43
XA_NORM_SPACE             44
XA_MAX_SPACE              45
XA_END_SPACE              46
XA_SUPERSCRIPT_X          47
XA_SUPERSCRIPT_Y          48
XA_SUBSCRIPT_X            49
XA_SUBSCRIPT_Y            50
XA_UNDERLINE_POSITION     51
XA_UNDERLINE_THICKNESS    52
XA_STRIKEOUT_ASCENT       53
XA_STRIKEOUT_DESCENT      54
XA_ITALIC_ANGLE           55
XA_X_HEIGHT               56
```

```
XA_QUAD_WIDTH             57
XA_WEIGHT                 58
XA_POINT_SIZE             59
XA_RESOLUTION             60
XA_COPYRIGHT              61
XA_NOTICE                 62
XA_FONT_NAME              63
XA_FAMILY_NAME            64
XA_FULL_NAME              65
XA_CAP_HEIGHT             66
XA_WM_CLASS               67
XA_WM_TRANSIENT_FOR       68
XA_LAST_PREDEFINED        68
```

- **Atom strings**:

```
"PRIMARY"
"SECONDARY"
"ARC"
"ATOM"
"BITMAP"
"CARDINAL"
"COLORMAP"
"CURSOR"
"CUT_BUFFER0"
"CUT_BUFFER1"
"CUT_BUFFER2"
"CUT_BUFFER3"
"CUT_BUFFER4"
"CUT_BUFFER5"
"CUT_BUFFER6"
"CUT_BUFFER7"
"DRAWABLE"
"FONT"
"INTEGER"
"PIXMAP"
"POINT"
"RECTANGLE"
```

```
"RESOURCE_MANAGER"
"RGB_COLOR_MAP"
"RGB_BEST_MAP"
"RGB_BLUE_MAP"
"RGB_DEFAULT_MAP"
"RGB_GRAY_MAP"
"RGB_GREEN_MAP"
"RGB_RED_MAP"
"STRING"
"VISUALID"
"WINDOW"
"WM_COMMAND"
"WM_HINTS"
"WM_CLIENT_MACHINE"
"WM_ICON_NAME"
"WM_ICON_SIZE"
"WM_NAME"
"WM_NORMAL_HINTS"
```

```
"WM_SIZE_HINTS"
"WM_ZOOM_HINTS"
"MIN_SPACE"
"NORM_SPACE"
"MAX_SPACE"
"END_SPACE"
"SUPERSCRIPT_X"
"SUPERSCRIPT_Y"
"SUBSCRIPT_X"
"SUBSCRIPT_Y"
"UNDERLINE_POSITION"
"UNDERLINE_THICKNESS"
"STRIKEOUT_ASCENT"
"STRIKEOUT_DESCENT"
"ITALIC_ANGLE"
```

```
"X_HEIGHT"
"QUAD_WIDTH"
"WEIGHT"
"POINT_SIZE"
"RESOLUTION"
"COPYRIGHT"
"NOTICE"
"FONT_NAME"
"FAMILY_NAME"
"FULL_NAME"
"CAP_HEIGHT"
"WM_CLASS"
"WM_TRANSIENT_FOR"
```

- **Prefixes**

```
XA_         // predefined atom macros
WM_         // ICCCM window-manager atoms/properties/protocols
_NET_       // EWMH / NetWM atoms
XK_         // KeySym constants
XC_         // cursor font constants
Xcms        // X Color Management System symbols
X           // Xlib function/type prefix
```

## Status return codes

```
#define Status int
```

- **Lookup status values**: Used by input-method lookup functions such as XmbLookupString, XwcLookupString, and related lookup APIs.

```
#define XBufferOverflow   -1
#define XLookupNone        1
#define XLookupChars       2
#define XLookupKeySym      3
#define XLookupBoth        4
```

- **Bitmap status values**: Used by bitmap-file functions such as XReadBitmapFile.

```
#define BitmapSuccess       0
#define BitmapOpenFailed    1
#define BitmapFileInvalid   2
#define BitmapNoMemory      3
```

- **Context manager status values**: Used by Xlib context-manager functions.

```
#define XCSUCCESS   0
#define XCNOMEM     1
#define XCNOENT     2
```

- **Locale / converter status values**: Used by locale and text conversion setup functions.

```
#define XNoMemory              -1
#define XLocaleNotSupported    -2
#define XConverterNotFound     -3
```

- **Grab status values**: Used by grab functions such as XGrabPointer and XGrabKeyboard.

```
#define GrabSuccess       0
#define AlreadyGrabbed    1
#define GrabInvalidTime   2
#define GrabNotViewable   3
#define GrabFrozen        4
```

## Keymask modifiers

- **Keymask modifiers**: In Xlib, keymask modifiers are bitwise constants used to describe the state of modifier keys (like Shift, Control, or Alt) during an input event. These masks are commonly found in the state member of event structures such as XKeyEvent or used as arguments in functions like **XGrabKey.**
  - **ShiftMask**: The Shift key is pressed.
  - **LockMask**: The Caps Lock key is active.
  - **ControlMask**: The Control key is pressed.
  - **Mod1Mask**: Usually mapped to Alt or Meta.
  - **Mod2Mask**: Often mapped to Num Lock.
  - **Mod3Mask**: Sometimes mapped to AltGr.
  - **Mod4Mask**: Commonly mapped to the Super/Windows key.
  - **Mod5Mask**: Often mapped to Scroll Lock or other custom modifiers.

# XIDs

- **XID definitions**

```
typedef XID Window;
typedef XID Drawable;
typedef XID Font;
typedef XID Pixmap;
typedef XID Cursor;
typedef XID Colormap;
typedef XID GContext;
typedef XID KeySym;
```

## Other typedefs

- **Other typedefs**

```
typedef unsigned long XID;
typedef unsigned long Mask;
typedef unsigned long Atom;
typedef unsigned char KeyCode;
typedef unsigned long VisualID;
typedef unsigned long Time;
```

# Cursors (<X11/cursorfont.h>)

- **Cursors**
  - **XC_X_cursor**: An "X" shaped cursor. Often used as a generic default or placeholder cursor.
  - **XC_arrow**: A generic arrow cursor.
  - **XC_based_arrow_down**: A downward-pointing arrow with a base/bar.
  - **XC_based_arrow_up**: An upward-pointing arrow with a base/bar.
  - **XC_boat**: A small boat-shaped cursor. Mostly decorative/legacy.
  - **XC_bogosity**: A strange "bad/invalid/weird" symbol. Mostly humorous/legacy.
  - **XC_bottom_left_corner**: Resize cursor for the bottom-left corner of a window.
  - **XC_bottom_right_corner**: Resize cursor for the bottom-right corner of a window.
  - **XC_bottom_side**: Resize cursor for the bottom edge of a window.
  - **XC_bottom_tee**: A T-shaped cursor associated with the bottom side.
  - **XC_box_spiral**: A box/square spiral shape. Mostly decorative.
  - **XC_center_ptr**: A pointer arrow whose hotspot is more centered than XC_left_ptr.
  - **XC_circle**: A circular cursor.
  - **XC_clock**: A clock-shaped cursor, usually indicating waiting/busy state.
  - **XC_coffee_mug**: A coffee mug cursor. Mostly decorative/legacy.
  - **XC_cross**: A simple cross-shaped cursor.
  - **XC_cross_reverse**: A reverse/inverted version of the cross cursor.
  - **XC_crosshair**: A precision crosshair cursor, often used for targeting or coordinate selection.
  - **XC_diamond_cross**: A cross combined with a diamond-like shape.
  - **XC_dot**: A small dot cursor.
  - **XC_dotbox**: A dot inside a box. Useful for precise pointing.
  - **XC_double_arrow**: A double-ended arrow, generally suggesting resizing or directional adjustment.
  - **XC_draft_large**: A large drafting/drawing-style cursor.
  - **XC_draft_small**: A smaller drafting/drawing-style cursor.
  - **XC_draped_box**: A box-like cursor with a draped/covered appearance. Mostly decorative.
  - **XC_exchange**: An exchange/swap cursor, suggesting replacement or swapping.
  - **XC_fleur**: A four-direction move cursor, like arrows pointing up/down/left/right.
  - **XC_gobbler**: A "gobbler"/Pac-Man-like decorative cursor.
  - **XC_gumby**: A Gumby-like figure cursor. Decorative/legacy.
  - **XC_hand1**: A hand cursor, usually pointing or selecting.
  - **XC_hand2**: Another hand cursor variant. Commonly used for links or clickable things.
  - **XC_heart**: A heart-shaped cursor.
  - **XC_icon**: An icon-like cursor, historically associated with window/icon manipulation.

- **XC_iron_cross**: A heavy cross shape resembling an iron cross.
- **XC_left_ptr**: The usual left-leaning arrow pointer. This is the common default pointer.
- **XC_left_side**: Resize cursor for the left edge of a window.
- **XC_left_tee**: A T-shaped cursor associated with the left side.
- **XC_leftbutton**: A mouse/button symbol emphasizing the left button.
- **XC_ll_angle**: Lower-left angle/corner shape.
- **XC_lr_angle**: Lower-right angle/corner shape.
- **XC_man**: A small human figure cursor.
- **XC_middlebutton**: A mouse/button symbol emphasizing the middle button.
- **XC_mouse**: A mouse-shaped cursor.
- **XC_pencil**: A pencil cursor, commonly used for drawing/editing.
- **XC_pirate**: A pirate/skull-style cursor, often used for destructive or dangerous actions.
- **XC_plus**: A plus-sign cursor. Often suggests adding, inserting, or selecting.
- **XC_question_arrow**: An arrow with a question mark, commonly used for help/context help.
- **XC_right_ptr**: A right-leaning or right-pointing pointer arrow.
- **XC_right_side**: Resize cursor for the right edge of a window.
- **XC_right_tee**: A T-shaped cursor associated with the right side.
- **XC_rightbutton**: A mouse/button symbol emphasizing the right button.
- **XC_rtl_logo**: A stylized logo cursor. Rarely used in modern programs.
- **XC_sailboat**: A sailboat-shaped cursor. Decorative/legacy.
- **XC_sb_down_arrow**: Scrollbar-style down arrow.
- **XC_sb_h_double_arrow**: Horizontal double arrow, commonly suggesting horizontal resizing or scrolling.
- **XC_sb_left_arrow**: Scrollbar-style left arrow.
- **XC_sb_right_arrow**: Scrollbar-style right arrow.
- **XC_sb_up_arrow**: Scrollbar-style up arrow.
- **XC_sb_v_double_arrow**: Vertical double arrow, commonly suggesting vertical resizing or scrolling.
- **XC_shuttle**: A space-shuttle-shaped cursor. Decorative/legacy.
- **XC_sizing**: Generic sizing/resizing cursor.
- **XC_spider**: Spider-shaped cursor. Decorative/legacy.
- **XC_spraycan**: Spray-can cursor, commonly associated with painting/drawing programs.
- **XC_star**: Star-shaped cursor.
- **XC_target**: Target/bullseye cursor.
- **XC_tcross**: A T-like cross cursor.
- **XC_top_left_arrow**: Arrow pointing toward the upper-left.
- **XC_top_left_corner**: Resize cursor for the top-left corner of a window.
- **XC_top_right_corner**: Resize cursor for the top-right corner of a window.
- **XC_top_side**: Resize cursor for the top edge of a window.

- **XC_top_tee**: A T-shaped cursor associated with the top side.
- **XC_trek**: A Star-Trek-like insignia cursor. Decorative/legacy.
- **XC_ul_angle**: Upper-left angle/corner shape.
- **XC_umbrella**: Umbrella-shaped cursor. Decorative/legacy.
- **XC_ur_angle**: Upper-right angle/corner shape.
- **XC_watch**: Watch cursor, commonly used to indicate busy/waiting.
- **XC_xterm**: Text insertion/I-beam cursor, commonly used over text areas.

The practical / common ones is the subset

| Cursor | Description | Common use |
|---|---|---|
| XC_left_ptr | Standard arrow pointer, usually angled up-left. | Default cursor for normal pointing and selection. |
| XC_xterm | Text insertion cursor, usually an I-beam. | Used over editable or selectable text regions. |
| XC_watch | Watch/busy cursor. | Indicates that the application is busy or waiting. |
| XC_hand1 | Hand-shaped cursor, first variant. | Used for clickable or draggable objects. |
| XC_hand2 | Hand-shaped cursor, second variant. | Commonly used for links or clickable UI elements. |
| XC_crosshair | Crosshair cursor with horizontal and vertical lines. | Precision pointing, drawing, targeting, coordinate selection. |
| XC_fleur | Four-direction movement cursor. | Moving windows, panels, objects, or selections. |
| XC_sb_h_double_arrow | Horizontal double-ended arrow. | Horizontal resizing or horizontal scrollbar interaction. |
| XC_sb_v_double_arrow | Vertical double-ended arrow. | Vertical resizing or vertical scrollbar interaction. |
| XC_top_side | Resize cursor for the top edge. | Resizing a window or object from its top border. |
| XC_bottom_side | Resize cursor for the bottom edge. | Resizing from the bottom border. |
| XC_left_side | Resize cursor for the left edge. | Resizing from the left border. |
| XC_right_side | Resize cursor for the right edge. | Resizing from the right border. |
| XC_top_left_corner | Diagonal resize cursor for the top-left corner. | Resizing from the upper-left corner. |
| XC_top_right_corner | Diagonal resize cursor for the top-right corner. | Resizing from the upper-right corner. |
| XC_bottom_left_corner | Diagonal resize cursor for the bottom-left corner. | Resizing from the lower-left corner. |
| XC_bottom_right_corner | Diagonal resize cursor for the bottom-right corner. | Resizing from the lower-right corner. |

- **Creating Cursors**:

```
Cursor XCreateFontCursor(Display *display, unsigned int
↪ shape)
```

Creates a cursor from one of the standard cursor glyphs defined by the X cursor font.

- *display*: Specifies the connection to the X server.
- *shape*: Specifies the shape of the cursor.
- **Possible errors**:
  - **BadAlloc**
  - **BadValue**

## Display functions

- **Structures:**

```
typedef struct {
    int depth;
    int bits_per_pixel;
    int scanline_pad;
} XPixmapFormatValues;
```

- **Functions**:

```
Display *XOpenDisplay(char* display_name)
```

  - **Description**: Opens a connection to the X server and returns a `Display*` used for nearly all later Xlib calls.
  - **Possible errors**:
    - *None*: This function does not generate X protocol errors.
    - Failure to open the display is indicated by a `NULL` return value.

**Note:** Passing NULL tells X to use the DISPLAY environment variable.

```
AllPlanes
unsigned long XAllPlanes()
```

  - **Description**: Returns a bit mask with all plane bits set; commonly used when copying or querying all drawable planes.
  - **Possible errors**:
    - *None*: This function does not generate X protocol errors.

```
BlackPixel(Display* display, int screen_number)
unsigned long XBlackPixel(Display* display, int
    ↪ screen_number)
```

  - **Description**: Returns the black pixel value for the specified screen's default colormap.
  - **Possible errors**:
    - *None*: This function does not generate X protocol errors.

```
WhitePixel(Display* display, int screen_number)
unsigned long XWhitePixel(Display* display, int
    ↪ screen_number)
```

  - **Description**: Returns the white pixel value for the specified screen's default colormap.
  - **Possible errors**:
    - *None*: This function does not generate X protocol errors.

```
ConnectionNumber(Display* display)
int XConnectionNumber(Display* display)
```

- **Description**: Returns the file descriptor used by the X server connection.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DefaultColormap(Display* display, int screen_number)
Colormap XDefaultColormap(Display* display, int
    ↪  screen_number)
```

- **Description**: Returns the default colormap for the specified screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DefaultDepth(Display* display, int screen_number)
int XDefaultDepth(Display* display, int screen_number)
```

- **Description**: Returns the default depth, in bits per pixel, for the specified screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
int* XListDepths(Display* display, int screen_number, int*
    ↪  count_return)
```

- **Description**: Returns the list of supported depths for the specified screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.
  - Failure is indicated by a `NULL` return value.

```
DefaultGC(Display* display, int screen_number)
GC XDefaultGC(Display* display, int screen_number)
```

- **Description**: Returns the default graphics context for the specified screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DefaultRootWindow(Display* display)
Window XDefaultRootWindow(Display* display)
```

- **Description**: Returns the root window of the default screen.
- **Possible errors**:

    * *None*: This function does not generate X protocol errors.

```
DefaultScreenOfDisplay(Display* display)
Screen* XDefaultScreenOfDisplay(Display* display)
```

- **Description**: Returns a pointer to the default `Screen` structure for the display.
- **Possible errors**:
    * *None*: This function does not generate X protocol errors.

```
ScreenOfDisplay(Display* display, int screen_number)
Screen* XScreenOfDisplay(Display* display, int screen_number)
```

- **Description**: Returns the `Screen` structure for the specified screen number.
- **Possible errors**:
    * *None*: This function does not generate X protocol errors.

```
DefaultScreen(Display* display)
int XDefaultScreen(Display* display)
```

- **Description**: Returns the default screen number for the display.
- **Possible errors**:
    * *None*: This function does not generate X protocol errors.

```
DefaultVisual(Display* display, int screen_number)
Visual* XDefaultVisual(Display* display, int screen_number)
```

- **Description**: Returns the default visual for the specified screen.
- **Possible errors**:
    * *None*: This function does not generate X protocol errors.

```
DisplayCells(Display* display, int screen_number)
int XDisplayCells(Display* display, int screen_number)
```

- **Description**: Returns the number of colormap cells in the default colormap of the screen.
- **Possible errors**:
    * *None*: This function does not generate X protocol errors.

```
DisplayPlanes(Display* display, int screen_number)
int XDisplayPlanes(Display* display, int screen_number)
```

- **Description**: Returns the number of bit planes supported by the specified screen.
- **Possible errors**:

    * *None*: This function does not generate X protocol errors.

```
DisplayString(Display* display)
char* XDisplayString(Display* display)
```

- **Description**: Returns the display string used to open the display connection.
- **Possible errors**:
    * *None*: This function does not generate X protocol errors.

```
long XExtendedMaxRequestSize(Display* display)
```

- **Description**: Returns the maximum request size supported through the extended big-requests mechanism.
- **Possible errors**:
    * *None*: This function does not generate X protocol errors.

```
long XMaxRequestSize(Display* display)
```

- **Description**: Returns the maximum request size supported by the X server for normal protocol requests.
- **Possible errors**:
    * *None*: This function does not generate X protocol errors.

```
LastKnownRequestProcessed(Display* display)
unsigned long XLastKnownRequestProcessed(Display* display)
```

- **Description**: Returns the sequence number of the last request known to have been processed by the server.
- **Possible errors**:
    * *None*: This function does not generate X protocol errors.

```
NextRequest(Display* display)
unsigned long XNextRequest(Display* display)
```

- **Description**: Returns the sequence number that will be assigned to the next X protocol request.
- **Possible errors**:
    * *None*: This function does not generate X protocol errors.

```
ProtocolVersion(Display* display)
int XProtocolVersion(Display* display)
```

- **Description**: Returns the major version of the X protocol supported by the server.

- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
ProtocolRevision(Display* display)
int XProtocolRevision(Display* display)
```

- **Description**: Returns the minor protocol revision supported by the server.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
QLength(Display* display)
int XQLength(Display* display)
```

- **Description**: Returns the number of events already queued in Xlib's event queue.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
RootWindow(Display* display, int screen_number)
Window XRootWindow(Display* display, int screen_number)
```

- **Description**: Returns the root window for the specified screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
ScreenCount(Display* display)
int XScreenCount(Display* display)
```

- **Description**: Returns the number of screens attached to the display.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
ServerVendor(Display* display)
char* XServerVendor(Display* display)
```

- **Description**: Returns a string identifying the X server vendor.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
VendorRelease(Display* display)
int XVendorRelease(Display* display)
```

- **Description**: Returns the vendor-specific release number of the X server.
- **Possible errors**:

    - *None*: This function does not generate X protocol errors.

```
XPixmapFormatValues* XListPixmapFormats(Display* display,
    ↪  int* count_return)
```

- **Description**: Returns the pixmap formats supported by the display, including depth, bits per pixel, and scanline padding.
- **Possible errors**:
    - *None*: This function does not generate X protocol errors.
    - Failure is indicated by a `NULL` return value.

```
ImageByteOrder(Display* display)
int XImageByteOrder(Display* display)
```

- **Description**: Returns the byte order used for images transferred between client and server.
- **Possible errors**:
    - *None*: This function does not generate X protocol errors.

```
BitmapUnit(Display* display)
int XBitmapUnit(Display* display)
```

- **Description**: Returns the scanline unit used for bitmap data.
- **Possible errors**:
    - *None*: This function does not generate X protocol errors.

```
BitmapBitOrder(Display* display)
int XBitmapBitOrder(Display* display)
```

- **Description**: Returns the bit order used inside bitmap units.
- **Possible errors**:
    - *None*: This function does not generate X protocol errors.

```
BitmapPad(Display* display)
int XBitmapPad(Display* display)
```

- **Description**: Returns the bitmap scanline padding requirement.
- **Possible errors**:
    - *None*: This function does not generate X protocol errors.

```
DisplayHeight(Display* display, int screen_number)
int XDisplayHeight(Display* display, int screen_number)
```

- **Description**: Returns the height of the specified screen in pixels.

- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DisplayHeightMM(Display* display, int screen_number)
int XDisplayHeightMM(Display* display, int screen_number)
```

- **Description**: Returns the physical height of the specified screen in millimeters.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DisplayWidth(Display* display, int screen_number)
int XDisplayWidth(Display* display, int screen_number)
```

- **Description**: Returns the width of the specified screen in pixels.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DisplayWidthMM(Display* display, int screen_number)
int XDisplayWidthMM(Display* display, int screen_number)
```

- **Description**: Returns the physical width of the specified screen in millimeters.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
BlackPixelOfScreen(Screen* screen)
unsigned long XBlackPixelOfScreen(Screen* screen)
```

- **Description**: Returns the black pixel value for the given screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
WhitePixelOfScreen(Screen* screen)
unsigned long XWhitePixelOfScreen(Screen* screen)
```

- **Description**: Returns the white pixel value for the given screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
CellsOfScreen(Screen* screen)
int XCellsOfScreen(Screen* screen)
```

- **Description**: Returns the number of colormap cells available on the screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DefaultColormapOfScreen(Screen* screen)
Colormap XDefaultColormapOfScreen(Screen* screen)
```

- **Description**: Returns the default colormap associated with the given screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DefaultDepthOfScreen(Screen* screen)
int XDefaultDepthOfScreen(Screen* screen)
```

- **Description**: Returns the default depth of the given screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DefaultGCOfScreen(Screen* screen)
GC XDefaultGCOfScreen(Screen* screen)
```

- **Description**: Returns the default graphics context for the given screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DefaultVisualOfScreen(Screen* screen)
Visual *XDefaultVisualOfScreen(Screen* screen)
```

- **Description**: Returns the default visual for the given screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DoesBackingStore(Screen* screen)
int XDoesBackingStore(Screen* screen)
```

- **Description**: Returns whether the screen supports backing-store preservation for obscured window contents.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DoesSaveUnders(Screen* screen)
Bool XDoesSaveUnders(Screen* screen)
```

- **Description**: Returns whether the screen supports save-unders for temporarily obscured areas.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
DisplayOfScreen(Screen* screen)
Display *XDisplayOfScreen(Screen* screen)
```

- **Description**: Returns the display connection associated with the given screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
int XScreenNumberOfScreen(Screen* screen)
```

- **Description**: Returns the screen number corresponding to the given `Screen*`.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
EventMaskOfScreen(Screen* screen)
long XEventMaskOfScreen(Screen* screen)
```

- **Description**: Returns the event mask selected on the root window of the given screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
WidthOfScreen(Screen* screen)
int XWidthOfScreen(Screen* screen)
```

- **Description**: Returns the screen width in pixels.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
HeightOfScreen(Screen* screen)
int XHeightOfScreen(Screen* screen)
```

- **Description**: Returns the screen height in pixels.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
WidthMMOfScreen(Screen* screen)
int XWidthMMOfScreen(Screen* screen)
```

- **Description**: Returns the physical screen width in millimeters.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
HeightMMOfScreen(Screen* screen)
int XHeightMMOfScreen(Screen* screen)
```

- **Description**: Returns the physical screen height in millimeters.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
MaxCmapsOfScreen(Screen* screen)
int XMaxCmapsOfScreen(Screen* screen)
```

- **Description**: Returns the maximum number of installed colormaps supported by the screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
MinCmapsOfScreen(Screen* screen)
int XMinCmapsOfScreen(Screen* screen)
```

- **Description**: Returns the minimum number of installed colormaps guaranteed by the screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
PlanesOfScreen(Screen* screen)
int XPlanesOfScreen(Screen* screen)
```

- **Description**: Returns the number of bit planes supported by the screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
RootWindowOfScreen(Screen* screen)
Window XRootWindowOfScreen(Screen* screen)
```

- **Description**: Returns the root window of the given screen.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
int XNoOp(Display* display)
```

- **Description**: Sends a no-operation request to the server, useful for forcing request sequencing without changing server state.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
int XFree(void* data)
```

- **Description**: Frees memory allocated by Xlib routines.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
int XCloseDisplay(Display* display)
```

- **Description**: Closes the connection to the X server and frees resources associated with the display.
- **Possible errors**:
  - `BadGC`

```
int XSetCloseDownMode(Display* display, int close_mode)
```

- **Description**: Sets what happens to the client's server-side resources when the client disconnects.
- **Possible errors**:
  - `BadValue`

```
void XLockDisplay(Display* display)
```

- **Description**: Locks the display connection for thread-safe access.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
void XUnlockDisplay(Display* display)
```

- **Description**: Unlocks a display connection previously locked with `XLockDisplay`.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
Status XInitThreads();
```

- **Description**: Initializes Xlib support for multithreaded programs.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.
- **Status identifiers**:
  - `0`: Initialization failed.
  - `nonzero`: Initialization succeeded.

```
typedef void(*XConnectionWatchProc) (
    Display* display,
    XPointer client_data,
    int fd,
    Bool opening,
    XPointer watch_data
)

Status XAddConnectionWatch(
    Display* display,
    XConnectionWatchProc procedure,
    XPointer client_data
)
```

- **Description**: Registers a callback that is called when Xlib opens or closes internal connection file descriptors.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.
- **Status identifiers**:
  - `0`: The procedure was not registered.
  - `nonzero`: The procedure was successfully registered.

```
void XRemoveConnectionWatch(
    Display* display,
    XConnectionWatchProc procedure,
    XPointer client_data
)
```

- **Description**: Removes a previously registered internal-connection watch callback.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
void XProcessInternalConnection(Display* display, int fd)
```

- **Description**: Tells Xlib to process activity on one of its internal connection file descriptors.
- **Possible errors**:
  - *None*: This function does not generate X protocol errors.

```
Status XInternalConnectionNumbers(
    Display* display,
    int** fd_return,
    int* count_return
)
```

- **Description**: Returns the internal connection file descriptors that Xlib needs the application to monitor.

- **Possible errors**:
  - *None*: This function does not generate X protocol errors.
- **Status identifiers**:
  - `0`: The list was not successfully allocated.
  - `nonzero`: The list was successfully allocated.

## Window functions

- **Structures**:

```
typedef struct {
    Visual *visual;
    VisualID visualid;
    int screen;
    int depth;
    int class;
    unsigned long red_mask;
    unsigned long green_mask;
    unsigned long blue_mask;
    int colormap_size;
    int bits_per_rgb;
} XVisualInfo;
```

```
/* Window attribute value mask bits */
#define CWBackPixmap        (1L<<0)
#define CWBackPixel         (1L<<1)
#define CWBorderPixmap      (1L<<2)
#define CWBorderPixel       (1L<<3)
#define CWBitGravity        (1L<<4)
#define CWWinGravity        (1L<<5)
#define CWBackingStore      (1L<<6)
#define CWBackingPlanes     (1L<<7)
#define CWBackingPixel      (1L<<8)
#define CWOverrideRedirect  (1L<<9)
#define CWSaveUnder         (1L<<10)
#define CWEventMask         (1L<<11)
#define CWDontPropagate     (1L<<12)
#define CWColormap          (1L<<13)
#define CWCursor            (1L<<14)
```

```c
typedef struct {
    Pixmap background_pixmap;       /* background, None, or ParentRelative */
    unsigned long background_pixel; /* background pixel */
    Pixmap border_pixmap;           /* border of the window or CopyFromParent */
    unsigned long border_pixel;     /* border pixel value */
    int bit_gravity;                /* one of bit gravity values */
    int win_gravity;                /* one of the window gravity values */
    int backing_store;              /* NotUseful, WhenMapped, Always */
    unsigned long backing_planes;   /* planes to be preserved if possible */
    unsigned long backing_pixel;    /* value to use in restoring planes */
    Bool save_under;                /* should bits under be saved? (popups) */
    long event_mask;                /* set of events that should be saved */
    long do_not_propagate_mask;     /* set of events that should not propagate */
    Bool override_redirect;         /* boolean value for override_redirect */
    Colormap colormap;              /* color map to be associated with window */
    Cursor cursor;                  /* cursor to be displayed (or None) */
} XSetWindowAttributes;
```

```c
/* Configure window value mask bits */
#define CWX                 (1<<0)
#define CWY                 (1<<1)
#define CWWidth             (1<<2)
#define CWHeight            (1<<3)
#define CWBorderWidth       (1<<4)
#define CWSibling           (1<<5)
#define CWStackMode         (1<<6)

/* Values */

typedef struct {
    int x, y;
    int width, height;
    int border_width;
    Window sibling;
    int stack_mode;
} XWindowChanges;
```

- **Functions**:

```
VisualID XVisualIDFromVisual(Visual* visual);
```

Returns the visual ID associated with the specified **Visual** structure.

**Possible errors:**

– None specified.

```
XVisualInfo* XGetVisualInfo(
    Display* display,
    long vinfo_mask,
    XVisualInfo* vinfo_template,
    int* nitems_return
);
```

Returns a list of visual information structures that match the supplied visual template and mask.

– vinfo_mask Specifies the visual mask value.

– vinfo_template Specifies the visual attributes that are to be used in matching the visual structures.

– nitems_return Returns the number of matching visual structures.

**Possible errors:**

– None specified.

```
Window XCreateWindow(
    Display* display,
    Window parent,
    int x, y,
    unsigned int width, height,
    unsigned int border_width,
    int depth,
    unsigned int class,
    Visual* visual,
    unsigned long valuemask,
    XSetWindowAttributes* attributes
)
```

Creates a new window with explicitly specified geometry, depth, class, visual, and window attributes.

– **display**: Specifies the connection to the X server.

– **parent**: Specifies the parent window.

– **x**, **y**: Specify the $x$ and $y$ coordinates, which are the top-left outside corner of the created window's borders and are relative to the inside of the parent window's borders.

- **width, height**: Specify the width and height, which are the created window's inside dimensions and do not include the created window's borders. The dimensions must be nonzero, or a **BadValue** error results.
- **border_width**: Specifies the width of the created window's border in pixels.
- **depth**: Specifies the window's depth. A depth of **CopyFromParent** means the depth is taken from the parent.
- **class**: Specifies the created window's class. You can pass **InputOutput, InputOnly,** or **CopyFromParent.** A class of **CopyFromParent** means the class is taken from the parent.
- **visual**: Specifies the visual type. A visual of **CopyFromParent** means the visual type is taken from the parent.
- **valuemask**: Specifies which window attributes are defined in the attributes argument. This mask is the bitwise inclusive OR of the valid attribute mask bits. If valuemask is zero, the attributes are ignored and are not referenced.
- **attributes**: Specifies the structure from which the values (as specified by the value mask) are to be taken. The value mask should have the appropriate bits set to indicate which attributes have been set in the structure.

**Possible errors:**

- **BadAlloc**
- **BadColor**
- **BadCursor**
- **BadMatch**
- **BadPixmap**
- **BadValue**
- **BadWindow**

```
Window XCreateSimpleWindow(
    Display* display,
    Window parent,
    int x, y,
    unsigned int width, height,
    unsigned int border_width,
    unsigned long border,
    unsigned long background
)
```

Creates a basic **InputOutput** window using the parent window's depth, visual, and class.

- **display**: Specifies the connection to the $X$ server.
- **parent**: Specifies the parent window.
- $x$**, y**: Specify the $x$ and $y$ coordinates, which are the top-left outside corner of the new window's borders and are relative to the inside of the parent window's borders.
- **width, height**: Specify the width and height, which are the created window's inside dimensions and do not include the created window's borders. The dimensions must be nonzero, or a **BadValue** error results.

- **border_width**: Specifies the width of the created window's border in pixels.
- **border**: Specifies the border pixel value of the window.
- **background**: Specifies the background pixel value of the window.

**Possible errors:**

- **BadAlloc**
- **BadMatch**
- **BadValue**
- **BadWindow**

```
XDestroyWindow(Display* display, Window w)
```

Destroys the specified window and all of its subwindows.

**Possible errors:**

- **BadWindow**

```
XDestroySubwindows(Display* display, Window w)
```

Destroys all inferior windows of the specified window without destroying the window itself.

**Possible errors:**

- **BadWindow**

```
XMapWindow(Display* display, w)
```

Maps the specified window, making it eligible to become visible on the screen.

**Possible errors:**

- **BadWindow**

```
XMapRaised(Display* display, w)
```

Maps the specified window and raises it to the top of the stacking order.

**Possible errors:**

- **BadWindow**

```
XMapSubwindows(Display* display, w)
```

Maps all subwindows of the specified window.

**Possible errors:**

- **BadWindow**

```
XUnmapWindow(Display* display, w)
```

Unmaps the specified window, making it no longer visible.

**Possible errors:**

- **BadWindow**

```
XUnmapSubwindows(Display* display, w)
```

Unmaps all subwindows of the specified window.

**Possible errors:**

- **BadWindow**

```
XConfigureWindow(Display* display, Window w, unsigned int
↪  value_mask, XWindowChanges* values)
```

Changes the size, position, border width, or stacking order of the specified window.

- value_mask Specifies which values are to be set using information in the values structure. This mask is the bitwise inclusive OR of the valid configure window values bits.
- values Specifies the **XWindowChanges** structure.

**Possible errors:**

- **BadMatch**
- **BadValue**
- **BadWindow**

```
XMoveWindow(Display* display, w, x, y)
```

Moves the specified window to a new position relative to its parent.

- $x$, $y$ Specify the $x$ and $y$ coordinates, which define the new location of the top-left pixel of the window's border or the window itself if it has no border.

**Possible errors:**

- **BadWindow**

```
XResizeWindow(Display* display, Window w, unsigned int
↪ width, unsigned int height)
```

Resizes the interior area of the specified window.

- width, height Specify the width and height, which are the interior dimensions of the window after the call completes.

**Possible errors:**

- **BadValue**
- **BadWindow**

```
XMoveResizeWindow(Display* display, Window w, int $x$, int
↪ y, unsigned int width, unsigned int height)
```

Moves and resizes the specified window in a single request.

- $x$, $y$ Specify the $x$ and $y$ coordinates, which define the new position of the window relative to its parent.
- width, height Specify the width and height, which define the interior size of the window.

**Possible errors:**

- **BadValue**
- **BadWindow**

```
XSetWindowBorderWidth(Display* display, Window w, unsigned
↪ int width)
```

Sets the border width of the specified window.

**Possible errors:**

- **BadWindow**

```
XRaiseWindow(Display* display, w)
```

Raises the specified window to the top of the stacking order.

**Possible errors:**

- **BadWindow**

```
XLowerWindow(Display* display, w)
```

Lowers the specified window to the bottom of the stacking order.

**Possible errors:**

- **BadWindow**

```
XCirculateSubwindows(Display* display, Window w, int
↪  direction)
```

Circulates the subwindows of the specified window upward or downward in the stacking order.

- direction Specifies the direction (up or down) that you want to circulate the window. You can pass **RaiseLowest** or **LowerHighest.**

**Possible errors:**

- **BadValue**
- **BadWindow**

```
XCirculateSubwindowsUp(Display* display, w)
```

Raises the lowest mapped subwindow of the specified window to the top of the stacking order.

**Possible errors:**

- **BadWindow**

```
XCirculateSubwindowsDown(Display* display, w)
```

Lowers the highest mapped subwindow of the specified window to the bottom of the stacking order.

**Possible errors:**

- **BadWindow**

```
XRestackWindows(Display* display, Window windows[], int
↪  nwindows);
```

Restacks the specified windows according to their order in the given array.

- windows Specifies an array containing the windows to be restacked.
- nwindows Specifies the number of windows to be restacked.

**Possible errors:**

- **BadMatch**
- **BadWindow**

```
XChangeWindowAttributes(Display* display, Window w, unsigned
↪ long valuemask, XSetWindowAttributes* attributes)
```

Changes selected attributes of an existing window.

- valuemask Specifies which window attributes are defined in the attributes argument. This mask is the bitwise inclusive OR of the valid attribute mask bits. If valuemask is zero, the attributes are ignored and are not referenced. The values and restrictions are the same as for XCreateWindow.
- attributes Specifies the structure from which the values(as specified by the value mask) are to be taken. The value mask should have the appropriate bits set to indicate which attributes have been set in the structure

**Possible errors:**

- **BadAccess**
- **BadColor**
- **BadCursor**
- **BadMatch**
- **BadPixmap**
- **BadValue**
- **BadWindow**

```
XSetWindowBackground(Display* display, Window w, unsigned
↪ long background_pixel)
```

Sets the background pixel value used when the window background is drawn.

- background_pixel Specifies the pixel that is to be used for the background.

**Possible errors:**

- **BadMatch**
- **BadWindow**

```
XSetWindowBackgroundPixmap(Display* display, Window w,
↪ Pixmap background_pixmap)
```

Sets the background pixmap used when the window background is drawn.

**Possible errors:**

- **BadMatch**
- **BadPixmap**
- **BadWindow**

```
XSetWindowBorder(Display* display, w, unsigned long
↪ border_pixel)
```

Sets the border pixel value of the specified window.

- border_pixel Specifies the entry in the colormap.

**Possible errors:**

- **BadMatch**
- **BadWindow**

```
XSetWindowBorderPixmap(Display* display, w, Pixmap
↪ border_pixmap)
```

Sets the border pixmap used to draw the specified window's border.

- border_pixmap Specifies the border pixmap or CopyFromParent.

**Possible errors:**

- **BadMatch**
- **BadPixmap**
- **BadWindow**

```
XSetWindowColormap(Display* display, w, Colormap colormap)
```

Sets the colormap associated with the specified window.

**Possible errors:**

- **BadColor**
- **BadMatch**
- **BadWindow**

```
XDefineCursor(Display* display, w, Cursor cursor)
```

Defines the cursor that appears when the pointer is inside the specified window.

- cursor Specifies the cursor that is to be displayed or None.

**Possible errors:**

- **BadCursor**
- **BadWindow**

```
XUndefineCursor(Display* display, w)
```

Removes the window's explicitly defined cursor so that the parent window's cursor is inherited.

**Possible errors:**

- **BadWindow**

## Window information functions

- **Structures**

```
typedef struct {
    int x, y;                       /* location of window */
    int width, height;              /* width and height of window */
    int border_width;               /* border width of window */
    int depth;                      /* depth of window */
    Visual *visual;                 /* the associated visual structure */
    Window root;                    /* root of screen containing window */
    int class;                      /* InputOutput, InputOnly*/
    int bit_gravity;                /* one of the bit gravity values */
    int win_gravity;                /* one of the window gravity values */
    int backing_store;              /* NotUseful, WhenMapped, Always */
    unsigned long backing_planes;   /* planes to be preserved if possible */
    unsigned long backing_pixel;    /* value to be used when restoring planes */
    Bool save_under;                /* boolean, should bits under be saved? */
    Colormap colormap;              /* color map to be associated with window */
    Bool map_installed;             /* boolean, is color map currently installed*/
    int map_state;                  /* IsUnmapped, IsUnviewable, IsViewable */
    long all_event_masks;           /* set of events all people have interest in*/
    long your_event_mask;           /* my event mask */
    long do_not_propagate_mask;     /* set of events that should not propagate */
    Bool override_redirect;         /* boolean value for override-redirect */
    Screen *screen;                 /* back pointer to correct screen */
} XWindowAttributes;
```

- **Functions**

```
Status XQueryTree(Display* display, Window w, Window* root_return, Window* parent_return, Window** children_return, unsigned int* nchildren_return)
```

Queries the window hierarchy for the specified window and returns its root, parent, children, and number of children.

- root_return Returns the root window.
- parent_return Returns the parent window.
- children_return Returns the list of children.
- nchildren_return Returns the number of children.
- **Possible errors**:
  - **BadWindow**
- **Status identifiers**:
  - **0**: Failure.
  - **Nonzero**: Success.

```
Status XGetWindowAttributes(display, w, XWindowAttributes*
↪  window_attributes_return)
```

Retrieves the current attributes of the specified window and stores them in an XWindowAttributes structure.

- **window_attributes_return**: Returns the specified window's attributes in the XWindowAttributes structure.
- **Possible errors**:
  - **BadDrawable**
  - **BadWindow**
- **Status identifiers**:
  - **0**: Failure.
  - **Nonzero**: Success.

```
Status XGetGeometry(
    Display* display,
    Drawable d,
    Window* root_return,
    int* x_return, int* y_return,
    unsigned int* width_return, unsigned int* height_return,
    unsigned int* border_width_return,
    unsigned int* depth_return
)
```

Gets the geometry of a drawable, including its root window, position, size, border width, and depth.

- d Specifies the drawable, which can be a window or a pixmap.
- root_return Returns the root window.
- x_return, y_return Return the $x$ and $y$ coordinates that define the location of the drawable. For a window, these coordinates specify the upper-left outer corner relative to its parent's origin. For pixmaps, these coordinates are always zero.

- width_return, height_return Return the drawable's dimensions (width and height). For a window, these dimensions specify the inside size, not including the border.
- border_width_return Returns the border width in pixels. If the drawable is a pixmap, it returns zero.
- depth_return Returns the depth of the drawable (bits per pixel for the object).
- **Possible errors**:
  - **BadDrawable**
- **Status identifiers**:
  - **0**: Failure.
  - **Nonzero**: Success.

```
    Bool XTranslateCoordinates(
        Display* display,
        Window src_w, Window dest_w,
        int src_x, int src_y,
        int* dest_x_return, int* dest_y_return,
        Window* child_return
)
```

Translates coordinates from one window's coordinate system into another window's coordinate system.

- display Specifies the connection to the X server.
- src_w Specifies the source window.
- dest_w Specifies the destination window.
- src_x, src_y Specify the $x$ and $y$ coordinates within the source window.
- dest_x_return, dest_y_return Return the $x$ and $y$ coordinates within the destination window.
- child_return Returns the child if the coordinates are contained in a mapped child of the destination window.
- **Possible errors**:
  - **BadWindow**

```
Bool XQueryPointer(
    Display* display,
    Window w,
    Window* root_return, Window* child_return,
    int* root_x_return, int* root_y_return,
    int* win_x_return, int* win_y_return,
    unsigned int* mask_return
)
```

Queries the current pointer position and button/modifier state relative to both the root window and the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.

- root_return Returns the root window that the pointer is in.
- child_return Returns the child window that the pointer is located in, if any.
- root_x_return, root_y_return Return the pointer coordinates relative to the root window's origin.
- win_x_return, win_y_return Return the pointer coordinates relative to the specified window.
- mask_return Returns the current state of the modifier keys and pointer buttons.
- **Possible errors**:
  - **BadWindow**

```
Atom XInternAtom(Display* display, char* atom_name, Bool
↪ only_if_exists)
```

Returns the atom identifier associated with the specified atom name, optionally creating it if it does not already exist.

- display Specifies the connection to the X server.
- atom_name Specifies the name associated with the atom you want returned.
- only_if_exists Specifies a Boolean value that indicates whether the atom must be created.
- **Possible errors**:
  - **BadAlloc**
  - **BadValue**

```
Status XInternAtoms(Display* display, char** names, int
↪ count, Bool only_if_exists, Atom* atoms_return)
```

Converts multiple atom names into their corresponding atom identifiers in a single call.

- display Specifies the connection to the X server.
- names Specifies the array of atom names.
- count Specifies the number of atom names in the array.
- only_if_exists Specifies a Boolean value that indicates whether the atom must be created.
- atoms_return Returns the atoms.
- **Possible errors**:
  - **BadAlloc**
  - **BadValue**
- **Status identifiers**:
  - **0**: Failure.
  - **Nonzero**: Success.

```
char *XGetAtomName(Display display, Atom atom)
```

Returns the string name associated with the specified atom.

- display Specifies the connection to the X server.
- atom Specifies the atom for the property name you want returned.
- **Possible errors**:
  - **BadAtom**

```
Status XGetAtomNames(Display* display, Atom* atoms, int
↪  count, char** names_return)
```

Returns the string names associated with an array of atom identifiers.

- display Specifies the connection to the X server.
- atoms Specifies the array of atoms.
- count Specifies the number of atoms in the array.
- names_return Returns the atom names.
- **Possible errors**:
  - **BadAtom**
- **Status identifiers**:
  - **0**: Failure.
  - **Nonzero**: Success.

```
int XGetWindowProperty(
    Display* display,
    Window w,
    Atom property,
    long long_offset, long_length,
    Bool delete,
    Atom req_type,
    Atom* actual_type_return,
    int* actual_format_return,
    unsigned long* nitems_return,
    unsigned long* bytes_after_return,
    unsigned char** prop_return
)
```

Retrieves data from a window property and returns its actual type, format, item count, remaining byte count, and property contents.

- display Specifies the connection to the X server.
- *w* Specifies the window whose property you want to obtain.
- property Specifies the property name.
- long_offset Specifies the offset in the specified property (in 32-bit quantities) where the data is to be retrieved.
- long_length Specifies the length in 32-bit multiples of the data to be retrieved.
- delete Specifies a Boolean value that determines whether the property is deleted.

- req_type Specifies the atom identifier associated with the property type or **AnyPropertyType**.
- actual_type_return Returns the atom identifier that defines the actual type of the property.
- actual_format_return Returns the actual format of the property.
- nitems_return Returns the actual number of 8-bit, 16-bit, or 32-bit items stored in the prop_return data.
- bytes_after_return Returns the number of bytes remaining to be read in the property if a partial read was performed.
- prop_return Returns the data in the specified format.

- **Possible errors**:
  - **BadAtom**
  - **BadValue**
  - **BadWindow**
- **Return identifiers**:
  - **Success**

```
Atom* XListProperties(Display* display, Window w, int*
↪  num_prop_return)
```

Returns the list of property atoms currently defined on the specified window.

- display Specifies the connection to the X server.
- *w* Specifies the window whose property list you want to obtain.
- num_prop_return Returns the length of the properties array.

- **Possible errors**:
  - **BadWindow**

```
XChangeProperty(
    Display* display,
    Window w,
    Atom property, type,
    int format,
    int mode,
    unsigned char* data,
    int nelements
)
```

Changes, replaces, prepends, or appends data to a property on the specified window.

- display Specifies the connection to the X server.
- *w* Specifies the window whose property you want to change.
- property Specifies the property name.
- type Specifies the type of the property. The X server does not interpret the type but simply passes it back to an application that later calls **XGetWindowProperty.**

- format Specifies whether the data should be viewed as a list of 8-bit, 16-bit, or 32-bit quantities. Possible values are 8, 16, and 32. This information allows the X server to correctly perform byte-swap operations as necessary. If the format is 16-bit or 32-bit, you must explicitly cast your data pointer to an (unsigned char *) in the call to **XChangeProperty.**
- mode Specifies the mode of the operation. You can pass **PropModeReplace, PropModePrepend,** or **PropModeAppend.**
- data Specifies the property data.
- nelements Specifies the number of elements of the specified data format.
- **Possible errors**:
  * **BadAlloc**
  * **BadAtom**
  * **BadMatch**
  * **BadValue**
  * **BadWindow**

```
XRotateWindowProperties(display, w, properties, num_prop,
↪ npositions)
```

Rotates the values of a set of properties on the specified window by the given number of positions.

- display Specifies the connection to the X server.
- *w* Specifies the window.
- properties Specifies the array of properties that are to be rotated.
- num_prop Specifies the length of the properties array.
- npositions Specifies the rotation amount.
- **Possible errors**:
  * **BadAtom**
  * **BadMatch**
  * **BadWindow**

```
XDeleteProperty(display, w, property)
```

Deletes the specified property from the specified window.

- display Specifies the connection to the X server.
- *w* Specifies the window whose property you want to delete.
- property Specifies the property name.
- **Possible errors**:
  * **BadAtom**
  * **BadWindow**

```
XSetSelectionOwner(Display* display, Atom selection, Window
↪ owner, Time time)
```

Sets the owner window for a selection, such as PRIMARY or CLIPBOARD.

- display Specifies the connection to the X server.
- selection Specifies the selection atom.
- owner Specifies the owner of the specified selection atom. You can pass a window or **None.**
- time Specifies the time. You can pass either a timestamp or **CurrentTime.**
- **Possible errors**:
  - **BadAtom**
  - **BadWindow**

```
XSetSelectionOwner(Display* display, Atom selection, Window
↪  owner, Time time)
```

Sets the owner window for a selection, replacing any previous owner for that selection.

- display Specifies the connection to the X server.
- selection Specifies the selection atom.
- owner Specifies the owner of the specified selection atom. You can pass a window or **None.**
- time Specifies the time. You can pass either a timestamp or **CurrentTime.**
- **Possible errors**:
  - **BadAtom**
  - **BadWindow**

```
Window XGetSelectionOwner(Display* display, Atom selection)
```

Returns the current owner window of the specified selection.

- display Specifies the connection to the X server.
- selection Specifies the selection atom whose owner you want returned.
- **Possible errors**:
  - **BadAtom**

```
XConvertSelection(
    Display* display,
    Atom selection,
    Atom target,
    Atom property,
    Window requestor,
    Time time
)
```

Requests conversion of a selection to a target format and asks the selection owner to store the result in a property on the requestor window.

- display Specifies the connection to the X server.
- selection Specifies the selection atom.
- target Specifies the target atom.
- property Specifies the property name. You also can pass None.
- requestor Specifies the requestor.
- time Specifies the time. You can pass either a timestamp or CurrentTime.
- **Possible errors**:
  - **BadAtom**
  - **BadWindow**

## Pixmap and cursor functions

- **Structures**:

```
typedef struct {
    unsigned long pixel;
    unsigned short red, green, blue;
    char flags;
    char pad;
} XColor;
```

- **Functions**:

```
Pixmap XCreatePixmap(
    Display* display,
    Drawable d,
    unsigned int width,
    unsigned int height,
    unsigned int depth
)
```

Creates an off-screen pixmap drawable with the specified dimensions and depth on the same screen as the given drawable.

  - *display*: Specifies the connection to the X server.
  - *d*: Specifies which screen the pixmap is created on.
  - *width*, *height*: Specify the width and height, which define the dimensions of the pixmap.
  - *depth*: Specifies the depth of the pixmap.
  - **Possible errors**:
    - **BadAlloc**
    - **BadDrawable**
    - **BadValue**

```
XFreePixmap(Display* display, Pixmap pixmap)
```

Frees the server-side storage associated with the specified pixmap.

  - *display*: Specifies the connection to the X server.
  - *pixmap*: Specifies the pixmap.
  - **Possible errors**:
    - **BadPixmap**

```
#include <X11/cursorfont.h>

Cursor XCreateFontCursor(Display *display, unsigned int
↪  shape)
```

Creates a cursor from one of the standard cursor glyphs defined by the X cursor font.

- *display*: Specifies the connection to the X server.
- *shape*: Specifies the shape of the cursor.
- **Possible errors**:
  - **BadAlloc**
  - **BadValue**

```
Cursor XCreateGlyphCursor(
    Display *display,
    Font source_font,
    Font mask_font,
    unsigned int source_char,
    unsigned int mask_char,
    XColor *foreground_color,
    XColor *background_color
)
```

Creates a cursor from glyphs in the specified source and mask fonts, using the supplied foreground and background colors.

- *display*: Specifies the connection to the X server.
- *source_font*: Specifies the font for the source glyph.
- *mask_font*: Specifies the font for the mask glyph, or **None**.
- *source_char*: Specifies the character glyph for the source.
- *mask_char*: Specifies the glyph character for the mask.
- *foreground_color*: Specifies the RGB values for the foreground of the source.
- *background_color*: Specifies the RGB values for the background of the source.
- **Possible errors**:
  - **BadAlloc**
  - **BadFont**
  - **BadValue**

```
Cursor XCreatePixmapCursor(
    Display* display,
    Pixmap source,
    Pixmap mask,
    XColor* foreground_color,
    XColor* background_color,
    unsigned int x,
    unsigned int y
)
```

Creates a cursor from a source pixmap and an optional mask pixmap, with the given hotspot coordinates.

*display*: Specifies the connection to the X server.

*source*: Specifies the shape of the source cursor.

*mask*: Specifies the cursor source bits to be displayed or **None**.

*foreground_color*: Specifies the RGB values for the foreground of the source.

*background_color*: Specifies the RGB values for the background of the source.

*x*, *y*: Specify the *x* and *y* coordinates, which indicate the hotspot relative to the source's origin.

- **Possible errors**:
  - **BadAlloc**
  - **BadMatch**
  - **BadPixmap**

```
Status XQueryBestCursor(
    Display* display,
    Drawable d,
    unsigned int width,
    unsigned int height,
    unsigned int* width_return,
    unsigned int* height_return
)
```

Queries the closest cursor size supported by the display for the specified drawable's screen.

- *display*: Specifies the connection to the X server.
- *d*: Specifies the drawable, which indicates the screen.
- *width*, *height*: Specify the width and height of the cursor that you want the size information for.
- *width_return*, *height_return*: Return the best width and height that is closest to the specified width and height.

- **Possible errors**:
  - **BadDrawable**
- **Possible Status returns**:
  - **nonzero**: Success.
  - **0**: Failure.

```
XRecolorCursor(
    Display* display,
    Cursor cursor,
    XColor* foreground_color,
    XColor* background_color
)
```

Changes the foreground and background colors of an existing cursor.

- *display*: Specifies the connection to the X server.
- *cursor*: Specifies the cursor.
- *foreground_color*: Specifies the RGB values for the foreground of the source.
- *background_color*: Specifies the RGB values for the background of the source.
- **Possible errors**:
  - **BadCursor**

```
XFreeCursor(Display* display, Cursor cursor)
```

Frees the server-side resources associated with the specified cursor.

- *display*: Specifies the connection to the X server.
- *cursor*: Specifies the cursor.
- **Possible errors**:
  - **BadCursor**

## Color management functions

- **Structures**:

```
typedef struct {
    unsigned long pixel; /* pixel value */
    unsigned short red, green, blue; /* rgb values */
    char flags; /* DoRed, DoGreen, DoBlue */
    char pad;
} XColor;
```

```
typedef struct {
    union {
        XcmsRGB RGB;
        XcmsRGBi RGBi;
        XcmsCIEXYZ CIEXYZ;
        XcmsCIEuvY CIEuvY;
        XcmsCIExyY CIExyY;
        XcmsCIELab CIELab;
        XcmsCIELuv CIELuv;
        XcmsTekHVC TekHVC;
        XcmsPad Pad;
    } spec;
    unsigned long pixel;
    XcmsColorFormat format;
} XcmsColor; /* Xcms Color Structure */
```

```
#define XcmsUndefinedFormat 0x00000000
#define XcmsCIEXYZFormat 0x00000001 /* CIE XYZ */
#define XcmsCIEuvYFormat 0x00000002 /* CIE u'v'Y */
#define XcmsCIExyYFormat 0x00000003 /* CIE xyY */
#define XcmsCIELabFormat 0x00000004 /* CIE L*a*b* */
#define XcmsCIELuvFormat 0x00000005 /* CIE L*u*v* */
#define XcmsTekHVCFormat 0x00000006 /* TekHVC */
#define XcmsRGBFormat 0x80000000 /* RGB Device */
#define XcmsRGBiFormat 0x80000001 /* RGB Intensity */
```

```
typedef double XcmsFloat;
typedef struct {
    unsigned short red; /* 0x0000 to 0xffff */
    unsigned short green; /* 0x0000 to 0xffff */
    unsigned short blue; /* 0x0000 to 0xffff */
} XcmsRGB; /* RGB Device */

typedef struct {
    XcmsFloat red; /* 0.0 to 1.0 */
    XcmsFloat green; /* 0.0 to 1.0 */
    XcmsFloat blue; /* 0.0 to 1.0 */
} XcmsRGBi; /* RGB Intensity */

typedef struct {
    XcmsFloat X;
    XcmsFloat Y; /* 0.0 to 1.0 */
    XcmsFloat Z;
} XcmsCIEXYZ; /* CIE XYZ */

typedef struct {
    XcmsFloat u_prime; /* 0.0 to ~0.6 */
    XcmsFloat v_prime; /* 0.0 to ~0.6 */
    XcmsFloat Y; /* 0.0 to 1.0 */
} XcmsCIEuvY; /* CIE u'v'Y */
```

```
typedef struct {
    XcmsFloat x; /* 0.0 to ~.75 */
    XcmsFloat y; /* 0.0 to ~.85 */
    XcmsFloat Y; /* 0.0 to 1.0 */
} XcmsCIExyY; /* CIE xyY */

typedef struct {
    XcmsFloat L_star; /* 0.0 to 100.0 */
    XcmsFloat a_star;
    XcmsFloat b_star;
} XcmsCIELab; /* CIE L*a*b* */

typedef struct {
    XcmsFloat L_star; /* 0.0 to 100.0 */
    XcmsFloat u_star;
    XcmsFloat v_star;
} XcmsCIELuv; /* CIE L*u*v* */

typedef struct {
    XcmsFloat H; /* 0.0 to 360.0 */
    XcmsFloat V; /* 0.0 to 100.0 */
    XcmsFloat C; /* 0.0 to 100.0 */
} XcmsTekHVC; /* TekHVC */

typedef struct {
    XcmsFloat pad0;
    XcmsFloat pad1;
    XcmsFloat pad2;
    XcmsFloat pad3;
} XcmsPad; /* four doubles */
```

- **Functions**:

```
Colormap XCreateColormap(
    Display *display,
    Window w,
    Visual *visual,
    int alloc
)
```

Creates a new colormap for the screen associated with the specified window, using the given visual and allocation policy.

  - **display**: Specifies the connection to the X server.
  - **w**: Specifies the window on whose screen you want to create a colormap.
  - **visual**: Specifies a visual type supported on the screen. If the visual type is not one supported by the screen, a **BadMatch** error results.
  - **alloc**: Specifies the colormap entries to be allocated. You can pass **AllocNone** or **AllocAll**.

**Errors**:

- **BadAlloc**
- **BadMatch**
- **BadValue**
- **BadWindow**

```
Colormap XCopyColormapAndFree(
    Display* display,
    Colormap colormap
)
```

Copies the specified colormap, moves any client-allocated read-only entries into the new colormap, and frees the corresponding entries in the original colormap.

- **display**: Specifies the connection to the X server.
- **colormap**: Specifies the colormap.

**Errors**:

- **BadAlloc**
- **BadColor**

```
XFreeColormap(Display *display, Colormap colormap)
```

Frees the specified colormap resource and releases its associated server-side storage when it is no longer needed.

- **display**: Specifies the connection to the X server.
- **colormap**: Specifies the colormap that you want to destroy.

**Errors**:

- **BadColor**

```
Status XLookupColor(
    Display *display,
    Colormap colormap,
    char *color_name,
    XColor *exact_def_return,
    XColor *screen_def_return
)
```

Looks up a named color and returns both its exact RGB definition and the closest color that can be represented on the target screen.

- **display**: Specifies the connection to the X server.
- **colormap**: Specifies the colormap.
- **color_name**: Specifies the color name string, for example, `red`, whose color definition structure you want returned.
- **exact_def_return**: Returns the exact RGB values.
- **screen_def_return**: Returns the closest RGB values provided by the hardware.

**Errors**:

- **BadColor**

**Status return values**:

- **Nonzero**: The color name was resolved.
- **Zero**: The color name was not resolved.

```
Status XParseColor(
    Display *display,
    Colormap colormap,
    char *spec,
    XColor *exact_def_return
)
```

Parses a color name or numeric color specification and fills an **XColor** structure with the exact color definition.

- **display**: Specifies the connection to the X server.
- **colormap**: Specifies the colormap.
- **spec**: Specifies the color name string; case is ignored.
- **exact_def_return**: Returns the exact color value for later use and sets the **DoRed**, **DoGreen**, and **DoBlue** flags.

**Errors**:

- **BadColor**

**Status return values**:

- **Nonzero**: The color specification was resolved.
- **Zero**: The color specification was not resolved.

```
Status XcmsLookupColor(
    Display *display,
    Colormap colormap,
    char *color_string,
    XcmsColor *color_exact_return,
    XcmsColor *color_screen_return,
    XcmsColorFormat result_format
)
```

Looks up a color using the X Color Management System and returns both the exact color and the closest screen-reproducible color in the requested format.

- **display**: Specifies the connection to the X server.
- **colormap**: Specifies the colormap.
- **color_string**: Specifies the color string.
- **color_exact_return**: Returns the color specification parsed from the color string or parsed from the corresponding string found in a color-name database.

- **color_screen_return**: Returns the color that can be reproduced on the screen.
- **result_format**: Specifies the color format for the returned color specifications, that is, the `color_screen_return` and `color_exact_return` arguments. If the format is **XcmsUndefinedFormat** and the color string contains a numerical color specification, the specification is returned in the format used in that numerical color specification. If the format is **XcmsUndefinedFormat** and the color string contains a color name, the specification is returned in the format used to store the color in the database.

**Errors**:

- **BadColor**
- **BadValue**

**Status identifiers**:

- **XcmsSuccess**
- **XcmsSuccessWithCompression**
- **XcmsFailure**

```
Status XAllocColor(
    Display *display,
    Colormap colormap,
    XColor *screen_in_out
)
```

Allocates a read-only color cell in the specified colormap that most closely matches the requested RGB values.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *screen_in_out*: Specifies and returns the values actually used in the colormap.

**Errors**:

- **BadColor**

**Status return values**:

- **Nonzero**: The allocation succeeded.
- **Zero**: The allocation failed.

```
Status XcmsAllocColor(
    Display *display,
    Colormap colormap,
    XcmsColor *color_in_out,
    XcmsColorFormat result_format
)
```

Allocates a color cell using an **XcmsColor** specification and returns the actual color and pixel value allocated in the colormap.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color_in_out*: Specifies the color to allocate and returns the pixel and color that is actually used in the colormap.
- *result_format*: Specifies the color format for the returned color specification.

**Errors**:

- **BadColor**

**Status identifiers**:

- **XcmsSuccess**
- **XcmsSuccessWithCompression**
- **XcmsFailure**

```
Status XAllocNamedColor(
    Display *display,
    Colormap colormap,
    char *color_name,
    XColor *screen_def_return,
    XColor *exact_def_return
)
```

Looks up a named color and allocates the closest available color cell for it in the specified colormap.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color_name*: Specifies the color name string, for example, `red`, whose color definition structure you want returned.
- *screen_def_return*: Returns the closest RGB values provided by the hardware.
- *exact_def_return*: Returns the exact RGB values.

**Errors**:

- **BadColor**

**Status return values**:

- **Nonzero**: A color cell was allocated.
- **Zero**: A color cell was not allocated.

```
Status XcmsAllocNamedColor(
    Display *display,
    Colormap colormap,
    char *color_string,
    XcmsColor *color_screen_return,
    XcmsColor *color_exact_return,
    XcmsColorFormat result_format
)
```

Allocates a named color through Xcms and returns both the exact color definition and the actual screen color allocated in the colormap.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color_string*: Specifies the color string whose color definition structure is to be returned.
- *color_screen_return*: Returns the pixel value of the color cell and color specification that actually is stored for that cell.
- *color_exact_return*: Returns the color specification parsed from the color string or parsed from the corresponding string found in a color-name database.
- *result_format*: Specifies the color format for the returned color specifications, namely the *color_screen_return* and *color_exact_return* arguments. If the format is **XcmsUndefinedFormat** and the color string contains a numerical color specification, the specification is returned in the format used in that numerical color specification. If the format is **XcmsUndefinedFormat** and the color string contains a color name, the specification is returned in the format used to store the color in the database.

**Errors**:

- **BadColor**

**Status identifiers**:

- **XcmsSuccess**
- **XcmsSuccessWithCompression**
- **XcmsFailure**

```
Status XAllocColorCells(
    Display *display,
    Colormap colormap,
    Bool contig,
    unsigned long plane_masks_return[],
    unsigned int nplanes,
    unsigned long pixels_return[],
    unsigned int npixels
)
```

Allocates writable color cells and optional plane masks, allowing the client to later store its own color values into those cells.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *contig*: Specifies a Boolean value that indicates whether the planes must be contiguous.
- *plane_masks_return*: Returns an array of plane masks.
- *nplanes*: Specifies the number of plane masks that are to be returned in the plane masks array.
- *pixels_return*: Returns an array of pixel values.
- *npixels*: Specifies the number of pixel values that are to be returned in the *pixels_return* array.

**Errors**:

- **BadColor**
- **BadValue**

**Status return values**:

- **Nonzero**: The allocation succeeded.
- **Zero**: The allocation failed.

```
Status XAllocColorPlanes(
    Display *display,
    Colormap colormap,
    Bool contig,
    unsigned long pixels_return[],
    int ncolors,
    int nreds,
    int ngreens,
    int nblues,
    unsigned long *rmask_return,
    unsigned long *gmask_return,
    unsigned long *bmask_return
)
```

Allocates writable color cells arranged into red, green, and blue planes, returning pixel values and component masks for direct color manipulation.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *contig*: Specifies a Boolean value that indicates whether the planes must be contiguous.
- *pixels_return*: Returns an array of pixel values. **XAllocColorPlanes** returns the pixel values in this array.
- *ncolors*: Specifies the number of pixel values that are to be returned in the *pixels_return* array.

- *nreds*: Specifies the number of red planes. The value you pass must be nonnegative.
- *ngreens*: Specifies the number of green planes. The value you pass must be nonnegative.
- *nblues*: Specifies the number of blue planes. The value you pass must be nonnegative.
- *rmask_return*: Returns the bit mask for the red planes.
- *gmask_return*: Returns the bit mask for the green planes.
- *bmask_return*: Returns the bit mask for the blue planes.

**Errors**:

- **BadColor**
- **BadValue**

**Status return values**:

- **Nonzero**: The allocation succeeded.
- **Zero**: The allocation failed.

```
int XFreeColors(
    Display *display,
    Colormap colormap,
    unsigned long pixels[],
    int npixels,
    unsigned long planes
)
```

Frees previously allocated color cells and plane combinations in the specified colormap.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *pixels*: Specifies an array of pixel values that map to the cells in the specified colormap.
- *npixels*: Specifies the number of pixels.
- *planes*: Specifies the planes you want to free.

**Errors**:

- **BadAccess**
- **BadColor**
- **BadValue**

```
int XStoreColor(
    Display *display,
    Colormap colormap,
    XColor *color
)
```

Stores a new RGB value into a writable color cell in the specified colormap.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color*: Specifies the pixel and RGB values.

**Errors**:

- **BadAccess**
- **BadColor**
- **BadValue**

```
int XStoreColors(
    Display *display,
    Colormap colormap,
    XColor color[],
    int ncolors
)
```

Stores multiple RGB color definitions into writable color cells in the specified colormap.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color*: Specifies an array of color definition structures to be stored.
- *ncolors*: Specifies the number of **XColor** structures in the color definition array.

**Errors**:

- **BadAccess**
- **BadColor**
- **BadValue**

```
Status XcmsStoreColor(
    Display *display,
    Colormap colormap,
    XcmsColor *color
)
```

Stores an Xcms color specification into a writable color cell in the specified colormap.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color*: Specifies the color cell and the color to store. Values specified in this **XcmsColor** structure remain unchanged on return.

**Errors**:

- **BadAccess**
- **BadColor**
- **BadValue**

**Status identifiers**:

- **XcmsSuccess**
- **XcmsSuccessWithCompression**
- **XcmsFailure**

```
Status XcmsStoreColors(
    Display *display,
    Colormap colormap,
    XcmsColor colors[],
    int ncolors,
    Bool compression_flags_return[]
)
```

Stores multiple Xcms color specifications into writable color cells and reports whether any requested colors were compressed.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *colors*: Specifies the color specification array of **XcmsColor** structures, each specifying a color cell and the color to store in that cell. Values specified in the array remain unchanged upon return.
- *ncolors*: Specifies the number of **XcmsColor** structures in the color-specification array.
- *compression_flags_return*: Returns an array of Boolean values indicating compression status. If a non-**NULL** pointer is supplied, each element of the array is set to **True** if the corresponding color was compressed and **False** otherwise. Pass **NULL** if the compression status is not useful.

**Errors**:

- **BadAccess**
- **BadColor**
- **BadValue**

**Status identifiers**:

- **XcmsSuccess**
- **XcmsSuccessWithCompression**
- **XcmsFailure**

```
int XStoreNamedColor(
    Display *display,
    Colormap colormap,
    char *color,
    unsigned long pixel,
    int flags
)
```

Looks up a named color and stores selected RGB components of that color into the specified writable colormap entry.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color*: Specifies the color name string, for example, `red`.
- *pixel*: Specifies the entry in the colormap.
- *flags*: Specifies which red, green, and blue components are set.

**Errors**:

- **BadAccess**
- **BadColor**
- **BadName**
- **BadValue**

```
int XQueryColor(
    Display *display,
    Colormap colormap,
    XColor *def_in_out
)
```

Queries the RGB value currently associated with a single pixel in the specified colormap.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *def_in_out*: Specifies and returns the RGB values for the pixel specified in the structure.

**Errors**:

- **BadColor**
- **BadValue**

```
int XQueryColors(
    Display *display,
    Colormap colormap,
    XColor defs_in_out[],
    int ncolors
)
```

Queries the RGB values currently associated with multiple pixels in the specified colormap.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *defs_in_out*: Specifies and returns an array of color definition structures for the pixels specified in the structures.
- *ncolors*: Specifies the number of **XColor** structures in the color definition array.

**Errors**:

- **BadColor**
- **BadValue**

```
Status XcmsQueryColor(
    Display *display,
    Colormap colormap,
    XcmsColor *color_in_out,
    XcmsColorFormat result_format
)
```

Queries a single colormap cell and returns its color specification in the requested Xcms color format.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *color_in_out*: Specifies the *pixel* member that indicates the color cell to query. The color specification stored for the color cell is returned in this **XcmsColor** structure.
- *result_format*: Specifies the color format for the returned color specification.

**Errors**:

- **BadColor**
- **BadValue**

**Status identifiers**:

- **XcmsSuccess**
- **XcmsFailure**

```
Status XcmsQueryColors(
    Display *display,
    Colormap colormap,
    XcmsColor colors_in_out[],
    unsigned int ncolors,
    XcmsColorFormat result_format
)
```

Queries multiple colormap cells and returns their color specifications in the requested Xcms color format.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *colors_in_out*: Specifies an array of **XcmsColor** structures, each *pixel* member indicating the color cell to query. The color specifications for the color cells are returned in these structures.
- *ncolors*: Specifies the number of **XcmsColor** structures in the color-specification array.
- *result_format*: Specifies the color format for the returned color specification.

**Errors**:

- **BadColor**
- **BadValue**

**Status identifiers**:

- **XcmsSuccess**
- **XcmsFailure**

## Graphics context functions

- **Structures**:
- **Functions**:

## Graphics functions

- **Structures**:
- **Functions**:

## Window and session manager functions

- **Functions**:

```
XReparentWindow(Display* display, Window w, Window parent,
↪  int x, int y)
```

Reparents the specified window so that it becomes a child of the specified parent window at the given position. The window is unmapped before the reparenting operation and then mapped again if it was originally mapped.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- parent Specifies the parent window.
- $x$, $y$ Specify the $x$ and $y$ coordinates of the position in the new parent window.

**Errors**:

- BadMatch
- BadWindow

```
XChangeSaveSet(Display* display, Window w, int change_mode)
```

Adds or removes the specified window from the client's save-set. A save-set is used so that windows created by other clients can be automatically restored if the managing client exits.

- display Specifies the connection to the X server.
- $w$ Specifies the window that you want to add to or delete from the client's save-set.
- change_mode Specifies the mode. You can pass SetModeInsert or SetModeDelete.

**Errors**:

- BadMatch
- BadValue
- BadWindow

```
XAddToSaveSet(Display* display, Window w)
```

Adds the specified window to the client's save-set. This is commonly used by window managers before reparenting client windows.

- display Specifies the connection to the X server.
- $w$ Specifies the window that you want to add to the client's save-set.

**Errors**:

- BadMatch
- BadWindow

```
XRemoveFromSaveSet(Display* display, Window w)
```

Removes the specified window from the client's save-set. After removal, the X server will no longer automatically restore that window if the client exits.

- display Specifies the connection to the X server.
- *w* Specifies the window that you want to delete from the client's save-set.

**Errors**:

- BadMatch
- BadWindow

```
XInstallColormap(Display* display, Colormap colormap)
```

Installs the specified colormap for its associated screen. Once installed, the colormap may be used by the server to interpret pixel values for windows using that colormap.

- display Specifies the connection to the X server.
- colormap Specifies the colormap.

**Errors**:

- BadColor

```
XUninstallColormap(Display* display, Colormap colormap)
```

Uninstalls the specified colormap from its associated screen. If possible, the server installs another suitable colormap after the specified one is removed.

- display Specifies the connection to the X server.
- colormap Specifies the colormap.

**Errors**:

- BadColor

```
Colormap *XListInstalledColormaps(Display* display, Window
↪  w, int* num_return)
```

Returns a list of the currently installed colormaps for the screen associated with the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window that determines the screen.
- num_return Returns the number of currently installed colormaps.

**Errors**:

- BadWindow

```
XSetFontPath(Display* display, char** directories, int ndirs)
```

Sets the server font search path. The X server uses this path when resolving font names requested by clients.

- display Specifies the connection to the X server.
- directories Specifies the directory path used to look for a font. Setting the path to the empty list restores the default path defined for the X server.
- ndirs Specifies the number of directories in the path.

**Errors**:

- BadValue

```
char** XGetFontPath(Display* display, int* npaths_return)
```

Returns the current font search path used by the X server. The returned list should be freed with XFreeFontPath when it is no longer needed.

- display Specifies the connection to the X server.
- npaths_return Returns the number of strings in the font path array.

**Errors**:

- No X protocol errors are documented for this function.

```
XFreeFontPath(char** list)
```

Frees the font path list returned by XGetFontPath.

- list Specifies the array of strings you want to free.

**Errors**:

- No X protocol errors are documented for this function.

```
XGrabServer(Display* display)
```

Grabs the X server so that no other client can process requests until the server is ungrabbed. This is useful when a client needs to perform a sequence of operations atomically with respect to other clients.

– display Specifies the connection to the X server.

**Errors**:

– No X protocol errors are documented for this function.

```
XUngrabServer(Display* display)
```

Releases a server grab previously established by XGrabServer. After this call, other clients may again process requests normally.

– display Specifies the connection to the X server.

**Errors**:

– No X protocol errors are documented for this function.

```
XKillClient(Display* display, XID resource)
```

Forces the client that owns the specified resource to close its connection to the X server. If AllTemporary is specified, all temporary resources are destroyed.

– display Specifies the connection to the X server.

– resource Specifies any resource associated with the client that you want to destroy or AllTemporary.

**Errors**:

– BadValue

```
XSetScreenSaver(Display* display, int timeout, int interval,
↪  int prefer_blanking, int allow_exposures)
```

Sets the screen saver parameters for the X server, including the timeout, interval, blanking preference, and exposure behavior.

– display Specifies the connection to the X server.

– timeout Specifies the timeout, in seconds, until the screen saver turns on.

– interval Specifies the interval, in seconds, between screen saver alterations.

– prefer_blanking Specifies how to enable screen blanking. You can pass DontPreferBlanking, PreferBlanking, or DefaultBlanking.

– allow_exposures Specifies the screen save control values. You can pass DontAllowExposures, AllowExposures, or DefaultExposures.

**Errors**:

- BadValue

```
XForceScreenSaver(Display* display, int mode)
```

Forces the screen saver either to activate immediately or to reset, depending on the specified mode.

- display Specifies the connection to the X server.
- mode Specifies the mode that is to be applied. You can pass ScreenSaverActive or ScreenSaverReset.

**Errors**:

- BadValue

```
XActivateScreenSaver(Display* display)
```

Activates the screen saver immediately. This is equivalent to forcing the screen saver into the active state.

- display Specifies the connection to the X server.

**Errors**:

- BadValue

```
XResetScreenSaver(Display* display)
```

Resets the screen saver timer as though user activity had occurred. This prevents the screen saver from activating until the timeout period elapses again.

- display Specifies the connection to the X server.

**Errors**:

- BadValue

```
XGetScreenSaver(
    Display* display,
    int* timeout_return,
    int* interval_return,
    int* prefer_blanking_return,
    int* allow_exposures_return
)
```

Returns the current screen saver settings for the X server, including the timeout, interval, blanking preference, and exposure behavior.

- display Specifies the connection to the X server.
- timeout_return Returns the timeout, in seconds, until the screen saver turns on.
- interval_return Returns the interval between screen saver invocations.
- prefer_blanking_return Returns the current screen blanking preference (DontPreferBlanking, PreferBlanking, or DefaultBlanking).
- allow_exposures_return Returns the current screen save control value (DontAllowExposures, AllowExposures, or DefaultExposures).

**Errors**:

- No X protocol errors are documented for this function.

# Events

- **Events**:

| Event Category | Event Type |
|---|---|
| Keyboard events | KeyPress, KeyRelease |
| Pointer events | ButtonPress, ButtonRelease, MotionNotify |
| Window crossing events | EnterNotify, LeaveNotify |
| Input focus events | FocusIn, FocusOut |
| Keymap state notification event | KeymapNotify |
| Exposure events | Expose, GraphicsExpose, NoExpose |
| Structure control events | CirculateRequest, ConfigureRequest, MapRequest, ResizeRequest |
| Window state notification events | CirculateNotify, ConfigureNotify, CreateNotify, DestroyNotify, GravityNotify, MapNotify, MappingNotify, ReparentNotify, UnmapNotify, VisibilityNotify |
| Colormap state notification event | ColormapNotify |
| Client communication events | ClientMessage, PropertyNotify, SelectionClear, SelectionNotify, SelectionRequest |

- **Keyboard and pointer events**
  - **KeyPress**: Generated when a key is pressed while the keyboard focus or pointer position selects the window.
  - **KeyRelease**: Generated when a key is released.
  - **ButtonPress**: Generated when a pointer button is pressed.
  - **ButtonRelease**: Generated when a pointer button is released.
  - **MotionNotify**: Generated when the pointer moves.
  - **EnterNotify**: Generated when the pointer enters a window.
  - **LeaveNotify**: Generated when the pointer leaves a window.
  - **KeymapNotify**: Reports the current keyboard state, usually after focus or crossing changes.
- **Keyboard and pointer masks**
  - **KeyPressMask**: Selects **KeyPress** events.
  - **KeyReleaseMask**: Selects **KeyRelease** events.
  - **ButtonPressMask**: Selects **ButtonPress** events.
  - **ButtonReleaseMask**: Selects **ButtonRelease** events.
  - **PointerMotionMask**: Selects pointer motion events.
  - **PointerMotionHintMask**: Requests pointer motion hints instead of every motion event.

  - **ButtonMotionMask**: Selects motion events while any pointer button is held.
  - **Button1MotionMask**: Selects motion events while button 1 is held.
  - **Button2MotionMask**: Selects motion events while button 2 is held.
  - **Button3MotionMask**: Selects motion events while button 3 is held.
  - **Button4MotionMask**: Selects motion events while button 4 is held.
  - **Button5MotionMask**: Selects motion events while button 5 is held.
  - **EnterWindowMask**: Selects **EnterNotify** events.
  - **LeaveWindowMask**: Selects **LeaveNotify** events.
  - **KeymapStateMask**: Selects **KeymapNotify** events.
- **Exposure and visibility events**
  - **Expose**: Generated when part of a window becomes visible and should be redrawn.
  - **VisibilityNotify**: Generated when a window's visibility state changes.
- **Exposure and visibility masks**
  - **ExposureMask**: Selects **Expose** events.
  - **VisibilityChangeMask**: Selects **VisibilityNotify** events.
- **Focus events**
  - **FocusIn**: Generated when a window receives keyboard focus.
  - **FocusOut**: Generated when a window loses keyboard focus.
- **Focus masks**
  - **FocusChangeMask**: Selects **FocusIn** and **FocusOut** events.
- **Window structure events**
  - **CreateNotify**: Generated when a child window is created.
  - **DestroyNotify**: Generated when a window is destroyed.
  - **UnmapNotify**: Generated when a window becomes unmapped.
  - **MapNotify**: Generated when a window becomes mapped.
  - **ReparentNotify**: Generated when a window is reparented.
  - **ConfigureNotify**: Generated when a window's size, position, border width, or stacking order changes.
  - **GravityNotify**: Generated when a window is moved because of bit gravity or window gravity.
  - **CirculateNotify**: Generated when a window is restacked to the top or bottom.
- **Window structure masks**

  - **StructureNotifyMask**: Selects structure events on the selected window itself.
  - **SubstructureNotifyMask**: Selects structure events involving child windows.

- **Redirect / request events**

  - **MapRequest**: Sent to a client that selected map redirection when another client requests that a child window be mapped.
  - **ConfigureRequest**: Sent to a client that selected configure redirection when another client requests a geometry or stacking change.
  - **CirculateRequest**: Sent to a client that selected circulation redirection when another client requests restacking by circulation.
  - **ResizeRequest**: Sent when a client requests that a window be resized and resize redirection is selected.

- **Redirect / request masks**

  - **SubstructureRedirectMask**: Selects redirection of map, configure, and circulate requests for child windows.
  - **ResizeRedirectMask**: Selects **ResizeRequest** events.

- **Property and colormap events**

  - **PropertyNotify**: Generated when a window property is changed or deleted.
  - **ColormapNotify**: Generated when a window's colormap changes or its install state changes.

- **Property and colormap masks**

  - **PropertyChangeMask**: Selects **PropertyNotify** events.
  - **ColormapChangeMask**: Selects **ColormapNotify** events.

- **Selection events cannot be selected directly**

  - **SelectionClear**: Sent to the previous selection owner when another client claims the selection.
  - **SelectionRequest**: Sent to the selection owner when another client requests selection data.
  - **SelectionNotify**: Sent to the requesting client when a selection conversion completes or fails.

- **Graphics operation events cannot be selected directly**

  - **GraphicsExpose**: Generated by certain graphics operations when exposed areas need to be redrawn.
  - **NoExpose**: Generated by certain graphics operations when no exposure resulted.

- **Client protocol events cannot be selected directly**

  - **ClientMessage**: Sent by clients using `XSendEvent`, commonly for protocols such as `WM_DELETE_WINDOW`.

- **Mapping events cannot be selected directly**

  - **MappingNotify**: Generated when the keyboard, pointer, or modifier mapping changes.

- **Unions**:

```c
typedef union _XEvent {
    int type;           /* must not be changed */
    XAnyEvent xany;
    XKeyEvent xkey;
    XButtonEvent xbutton;
    XMotionEvent xmotion;
    XCrossingEvent xcrossing;
    XFocusChangeEvent xfocus;
    XExposeEvent xexpose;
    XGraphicsExposeEvent xgraphicsexpose;
    XNoExposeEvent xnoexpose;
    XVisibilityEvent xvisibility;
    XCreateWindowEvent xcreatewindow;
    XDestroyWindowEvent xdestroywindow;
    XUnmapEvent xunmap;
    XMapEvent xmap;
    XMapRequestEvent xmaprequest;
    XReparentEvent xreparent;
    XConfigureEvent xconfigure;
    XGravityEvent xgravity;
    XResizeRequestEvent xresizerequest;
    XConfigureRequestEvent xconfigurerequest;
    XCirculateEvent xcirculate;
    XCirculateRequestEvent xcirculaterequest;
    XPropertyEvent xproperty;
    XSelectionClearEvent xselectionclear;
    XSelectionRequestEvent xselectionrequest;
    XSelectionEvent xselection;
    XColormapEvent xcolormap;
    XClientMessageEvent xclient;
    XMappingEvent xmapping;
    XErrorEvent xerror;
    XKeymapEvent xkeymap;
    long pad[24];
} XEvent;
```

- **Structures**:

```
typedef struct {
    int type;
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;
} XAnyEvent;
```

```
typedef struct {
    int type;               /* ButtonPress or ButtonRelease
    ↪  */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;          /* ``event'' window it is
    ↪  reported relative to */
    Window root;            /* root window that the event
    ↪  occurred on */
    Window subwindow;       /* child window */
    Time time;              /* milliseconds */
    int x, y;               /* pointer x, y coordinates in
    ↪  event window */
    int x_root, y_root;     /* coordinates relative to root
    ↪  */
    unsigned int state;     /* key or button mask */
    unsigned int button;    /* detail */
    Bool same_screen;       /* same screen flag */
} XButtonEvent;
typedef XButtonEvent XButtonPressedEvent;
typedef XButtonEvent XButtonReleasedEvent;
```

```
typedef struct {
    int type;               /* KeyPress or KeyRelease */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;          /* ``event'' window it is
    ↪  reported relative to */
    Window root;            /* root window that the event
    ↪  occurred on */
    Window subwindow;       /* child window */
    Time time;              /* milliseconds */
    int x, y;               /* pointer x, y coordinates in
    ↪  event window */
    int x_root, y_root;     /* coordinates relative to root
    ↪  */
    unsigned int state;     /* key or button mask */
    unsigned int keycode;   /* detail */
    Bool same_screen;       /* same screen flag */
} XKeyEvent;
typedef XKeyEvent XKeyPressedEvent;
typedef XKeyEvent XKeyReleasedEvent;
```

```
typedef struct {
    int type;               /* MotionNotify */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;          /* ``event'' window reported
    ↪  relative to */
    Window root;            /* root window that the event
    ↪  occurred on */
    Window subwindow;       /* child window */
    Time time;              /* milliseconds */
    int x, y;               /* pointer x, y coordinates in
    ↪  event window */
    int x_root, y_root;     /* coordinates relative to root
    ↪  */
    unsigned int state;     /* key or button mask */
    char is_hint;           /* detail */
    Bool same_screen;       /* same screen flag */
} XMotionEvent;
typedef XMotionEvent XPointerMovedEvent;
```

```
typedef struct {
    int type;               /* EnterNotify or LeaveNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window window;          /* ``event'' window reported
    ↪ relative to */
    Window root;            /* root window that the event
    ↪ occurred on */
    Window subwindow;       /* child window */
    Time time;              /* milliseconds */
    int x, y;               /* pointer x, y coordinates in
    ↪ event window */
    int x_root, y_root;     /* coordinates relative to root
    ↪ */
    int mode;               /* NotifyNormal, NotifyGrab,
    ↪ NotifyUngrab */
    int detail;
                            /*
                            * NotifyAncestor, NotifyVirtual,
                            ↪ NotifyInferior,
                            * NotifyNonlinear,NotifyNonlinea
                            ↪ rVirtual
                            */
    Bool same_screen;       /* same screen flag */
    Bool focus;             /* boolean focus */
    unsigned int state;     /* key or button mask */
} XCrossingEvent;
typedef XCrossingEvent XEnterWindowEvent;
typedef XCrossingEvent XLeaveWindowEvent;
```

```
typedef struct {
    int type;               /* FocusIn or FocusOut */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;          /* window of event */
    int mode;               /* NotifyNormal, NotifyGrab,
    ↪  NotifyUngrab */
    int detail;
    /*
    * NotifyAncestor, NotifyVirtual, NotifyInferior,
    * NotifyNonlinear,NotifyNonlinearVirtual, NotifyPointer,
    * NotifyPointerRoot, NotifyDetailNone
    */
} XFocusChangeEvent;
typedef XFocusChangeEvent XFocusInEvent;
typedef XFocusChangeEvent XFocusOutEvent;
```

```
/* generated on EnterWindow and FocusIn when KeymapState
↪  selected */
typedef struct {
    int type;                   /* KeymapNotify */
    unsigned long serial;       /* # of last request
    ↪  processed by server */
    Bool send_event;            /* true if this came from a
    ↪  SendEvent request */
    Display *display;           /* Display the event was read
    ↪  from */
    Window window;
    char key_vector[32];
} XKeymapEvent;
```

```
typedef struct {
    int type;               /* Expose */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;
    int x, y;
    int width, height;
    int count;              /* if nonzero, at least this many
    ↪  more */
} XExposeEvent;
```

```
typedef struct {
    int type;               /* GraphicsExpose */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Drawable drawable;
    int x, y;
    int width, height;
    int count;              /* if nonzero, at least this many
    ↪ more */
    int major_code;         /* core is CopyArea or CopyPlane
    ↪ */
    int minor_code;         /* not defined in the core */
} XGraphicsExposeEvent;

typedef struct {
    int type;               /* NoExpose */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Drawable drawable;
    int major_code;         /* core is CopyArea or CopyPlane
    ↪ */
    int minor_code;         /* not defined in the core */
} XNoExposeEvent;
```

```
typedef struct {
    int type;               /* CirculateNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window event;
    Window window;
    int place;              /* PlaceOnTop, PlaceOnBottom */
} XCirculateEvent;
```

```
typedef struct {
    int type;               /* ConfigureNotify */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window event;
    Window window;
    int x, y;
    int width, height;
    int border_width;
    Window above;
    Bool override_redirect;
} XConfigureEvent;
```

```
typedef struct {
    int type;                   /* CreateNotify */
    unsigned long serial;       /* # of last request
    ↪  processed by server */
    Bool send_event;            /* true if this came from a
    ↪  SendEvent request */
    Display *display;           /* Display the event was read
    ↪  from */
    Window parent;              /* parent of the window */
    Window window;              /* window id of window
    ↪  created */
    int x, y;                   /* window location */
    int width, height;          /* size of window */
    int border_width;           /* border width */
    Bool override_redirect;     /* creation should be
    ↪  overridden */
} XCreateWindowEvent;
```

```
typedef struct {
    int type; /* DestroyNotify */
    unsigned long serial; /* # of last request processed by
    ↪  server */
    Bool send_event; /* true if this came from a SendEvent
    ↪  request */
    Display *display; /* Display the event was read from */
    Window event;
    Window window;
} XDestroyWindowEvent;
```

```
typedef struct {
    int type;               /* GravityNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window event;
    Window window;
    int x, y;
} XGravityEvent;
```

```
typedef struct {
    int type;                   /* MapNotify */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window event;
    Window window;
    Bool override_redirect;     /* boolean, is override
    ↪ set... */
} XMapEvent;
```

```
typedef struct {
    int type;               /* UnmapNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window event;
    Window window;
    Bool from_configure;
} XUnmapEvent;
```

```
typedef struct {
    int type;               /* MappingNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window window;          /* unused */
    int request;            /* one of MappingModifier,
    ↪ MappingKeyboard,
    MappingPointer */
    int first_keycode;      /* first keycode */
    int count;              /* defines range of change w.
    ↪ first_keycode*/
} XMappingEvent;
```

```
typedef struct {
    int type;               /* ReparentNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window event;
    Window window;
    Window parent;
    int x, y;
    Bool override_redirect;
} XReparentEvent;
```

```
typedef struct {
    int type;               /* VisibilityNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window window;
    int state;
} XVisibilityEvent;
```

```
typedef struct {
    int type;                   /* CirculateRequest */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window parent;
    Window window;
    int place;                  /* PlaceOnTop, PlaceOnBottom
    ↪ */
} XCirculateRequestEvent;
```

```
typedef struct {
    int type;                   /* ConfigureRequest */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window parent;
    Window window;
    int x, y;
    int width, height;
    int border_width;
    Window above;
    int detail; /* Above, Below, TopIf, BottomIf, Opposite */
    unsigned long value_mask;
} XConfigureRequestEvent;
```

```
typedef struct {
    int type; /* MapRequest */
    unsigned long serial; /* # of last request processed by
    ↪ server */
    Bool send_event; /* true if this came from a SendEvent
    ↪ request */
    Display *display; /* Display the event was read from */
    Window parent;
    Window window;
} XMapRequestEvent;
```

```
typedef struct {
    int type;                   /* ResizeRequest */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window window;
    int width, height;
} XResizeRequestEvent;
```

```
typedef struct {
    int type;                   /* ColormapNotify */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window window;
    Colormap colormap;          /* colormap or None */
    Bool new;
    int state;                  /* ColormapInstalled,
    ↪ ColormapUninstalled */
} XColormapEvent;
```

```
typedef struct {
    int type;                   /* ClientMessage */
    unsigned long serial;       /* # of last request
    ↪ processed by server */
    Bool send_event;            /* true if this came from a
    ↪ SendEvent request */
    Display *display;           /* Display the event was read
    ↪ from */
    Window window;
    Atom message_type;
    int format;
    union {
        char b[20];
        short s[10];
        long l[5];
    } data;
} XClientMessageEvent;
```

```
typedef struct {
    int type;               /* PropertyNotify */
    unsigned long serial;   /* # of last request processed by
    ↪  server */
    Bool send_event;        /* true if this came from a
    ↪  SendEvent request */
    Display *display;       /* Display the event was read
    ↪  from */
    Window window;
    Atom atom;
    Time time;
    int state;              /* PropertyNewValue or
    ↪  PropertyDelete */
} XPropertyEvent;
```

```
typedef struct {
    int type;                   /* SelectionClear */
    unsigned long serial;       /* # of last request
    ↪  processed by server */
    Bool send_event;            /* true if this came from a
    ↪  SendEvent request */
    Display *display;           /* Display the event was read
    ↪  from */
    Window window;
    Atom selection;
    Time time;
} XSelectionClearEvent;
```

```
typedef struct {
    int type;                   /* SelectionRequest */
    unsigned long serial;       /* # of last request
    ↪  processed by server */
    Bool send_event;            /* true if this came from a
    ↪  SendEvent request */
    Display *display;           /* Display the event was read
    ↪  from */
    Window owner;
    Window requestor;
    Atom selection;
    Atom target;
    Atom property;
    Time time;
} XSelectionRequestEvent;
```

```
typedef struct {
    int type;               /* SelectionNotify */
    unsigned long serial;   /* # of last request processed by
    ↪ server */
    Bool send_event;        /* true if this came from a
    ↪ SendEvent request */
    Display *display;       /* Display the event was read
    ↪ from */
    Window requestor;
    Atom selection;
    Atom target;
    Atom property;          /* atom or None */
    Time time;
} XSelectionEvent;
```

## 17.1 Structure events

- **Structure events**: structure events are about changes to a window's position, size, mapping state, parent, stacking order, or lifetime.

  If you select

  ```
  XSelectInput(dpy, w, StructureNotifyMask);
  ```

  you are asking: Tell me when $w$ itself changes structurally.

  If you select

  ```
  XSelectInput(dpy, w, SubstructureNotifyMask);
  ```

  you are asking: Tell me when one of $w$'s direct child windows changes structurally.

  With

  ```
  XSelectInput(
      dpy,
      root,
      SubstructureNotifyMask | SubstructureRedirectMask
  );
  ```

  SubstructureNotifyMask on the root means: Tell me when top-level windows under the root are created, mapped, unmapped, configured, destroyed, reparented, etc.

  SubstructureRedirectMask tells us if another client tried to do something to a child window; the server stopped it and is asking you, usually the window manager, what to do.

- **StructureNotifyMask and SubStructureNotify**: Selecting this mask lets us receive the events

  ```
  MapNotify
  UnmapNotify
  DestroyNotify
  ConfigureNotify
  ReparentNotify
  GravityNotify
  CirculateNotify
  ```

- **Note about ResizeRedirectMask**: ResizeRedirectMask is selected on a window itself, not on its parent's substructure. It is for intercepting resize attempts on the selected window itself and receiving `ResizeRequest`.

  However, selecting SubstructureNotifyMask on the root will handle the case when top level children want to resize, so we don't need to select it.

## Event handling functions

- **Structures**:

```c
typedef struct {
    Time time;
    short x, y;
} XTimeCoord;
```

```c
typedef struct {
    int type;
    Display *display;                   /* Display the event
    ↪  was read from */
    unsigned long serial;               /* serial number of
    ↪  failed request */
    unsigned char error_code;           /* error code of
    ↪  failed request */
    unsigned char request_code;         /* Major op-code of
    ↪  failed request */
    unsigned char minor_code;           /* Minor op-code of
    ↪  failed request */
    XID resourceid;                     /* resource id */
} XErrorEvent;
```

- **Functions**:

```c
XSelectInput(Display* display, Window w, long event_mask)
```

Selects which input events the client wants to receive for the specified window.

  - **Possible errors**:
    * BadAccess
    * BadWindow

```c
XFlush(Display* display);
```

Flushes the output buffer, forcing all pending requests for the display connection to be sent to the X server.

  - **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported after the flush.

```c
XSync(Display* display, Bool discard);
```

Flushes the output buffer and waits until all requests have been processed by the X server.

  - **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be handled by the installed error handler.

```
int XEventsQueued(Display* display, int mode)
```

Returns the number of events already queued for the display connection, optionally checking the server depending on the specified mode.

- **Possible errors**: None documented.

```
int XPending(Display* display);
```

Returns the number of events already in the event queue after flushing the output buffer.

- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported after the flush.

```
XNextEvent(Display* display, XEvent* event_return)
```

Removes the next event from the event queue and copies it into the supplied event structure. If the queue is empty, it blocks until an event is available.

- display Specifies the connection to the X server.
- event_return Returns the next event in the queue.
- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported if the function flushes while waiting.

```
XPeekEvent(Display* display, XEvent* event_return)
```

Copies the next event from the event queue without removing it. If the queue is empty, it blocks until an event is available.

- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported if the function flushes while waiting.

```
Bool (*predicate)(Display* display, XEvent* event, XPointer
↪  arg)
```

Defines the predicate callback used by event-searching functions to decide whether an event matches the desired condition.

- display Specifies the connection to the X server.
- event Specifies the XEvent structure.
- arg Specifies the argument passed in from the XIfEvent, XCheckIfEvent, or XPeekIfEvent function.
- **Possible errors**: None documented.

```
XIfEvent(
    Display* display,
    XEvent* event_return,
    predicate,
    XPointer arg
)
```

Searches the event queue for an event that satisfies the predicate procedure, removes the matching event, and returns it.

- display Specifies the connection to the X server.
- event_return Returns the matched event's associated structure.
- predicate Specifies the procedure that is to be called to determine if the next event in the queue matches what you want.
- arg Specifies the user-supplied argument that will be passed to the predicate procedure.
- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported if the function flushes while waiting.

```
Bool XCheckIfEvent(Display* display, XEvent* event_return,
    ↪ predicate, XPointer arg)
```

Checks the event queue for an event that satisfies the predicate procedure and removes the matching event if one is found.

- display Specifies the connection to the X server.
- event_return Returns a copy of the matched event's associated structure.
- predicate Specifies the procedure that is to be called to determine if the next event in the queue matches what you want.
- arg Specifies the user-supplied argument that will be passed to the predicate procedure.
- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported if the function flushes.

```
XPeekIfEvent(Display* display, XEvent* event_return,
    ↪ predicate, XPointer arg)
```

Searches the event queue for an event that satisfies the predicate procedure and returns a copy without removing it from the queue.

- display Specifies the connection to the X server.
- event_return Returns a copy of the matched event's associated structure.
- predicate Specifies the procedure that is to be called to determine if the next event in the queue matches what you want.
- arg Specifies the user-supplied argument that will be passed to the predicate procedure.

- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported if the function flushes while waiting.

```
0 XWindowEvent(Display* display, Window w, long event_mask,
  ↪ XEvent* event_return)
```

Removes the next event for the specified window that matches the event mask, blocking until such an event is available.

- display Specifies the connection to the X server.
- $w$ Specifies the window whose events you are interested in.
- event_mask Specifies the event mask.
- event_return Returns the matched event's associated structure.
- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported if the function flushes while waiting.

```
0 Bool XCheckWindowEvent(Display* display, Window w, long
  ↪ event_mask, XEvent* event_return)
```

Checks whether an event for the specified window matches the event mask and removes it if one is found.

- display Specifies the connection to the X server.
- $w$ Specifies the window whose events you are interested in.
- event_mask Specifies the event mask.
- event_return Returns the matched event's associated structure.
- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported if the function flushes.

```
0 XMaskEvent(Display* display, long event_mask, XEvent*
  ↪ event_return)
```

Removes the next event matching the specified event mask, blocking until such an event is available.

- display Specifies the connection to the X server.
- event_mask Specifies the event mask.
- event_return Returns the matched event's associated structure.
- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported if the function flushes while waiting.

```
0 Bool XCheckMaskEvent(Display* display, long event_mask,
  ↪ XEvent* event_return)
```

Checks the event queue for an event matching the specified event mask and removes it if one is found.

- display Specifies the connection to the X server.
- event_mask Specifies the event mask.
- event_return Returns the matched event's associated structure.
- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported if the function flushes.

```
Bool XCheckTypedEvent(Display* display, int event_type,
↪  XEvent* event_return)
```

Checks the event queue for an event with the specified event type and removes it if one is found.

- display Specifies the connection to the X server.
- event_type Specifies the event type to be compared.
- event_return Returns the matched event's associated structure.
- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported if the function flushes.

```
Bool XCheckTypedWindowEvent(Display* display, Window w, int
↪  event_type, XEvent* event_return)
```

Checks the event queue for an event of the specified type associated with the specified window and removes it if one is found.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- event_type Specifies the event type to be compared.
- event_return Returns the matched event's associated structure.
- **Possible errors**: None documented for the call itself. Errors from previously buffered requests may be reported if the function flushes.

```
XPutBackEvent(Display* display, XEvent* event)
```

Pushes an event back onto the front of the event queue so it can be read again by later event-processing calls.

- display Specifies the connection to the X server.
- event Specifies the event.
- **Possible errors**: None documented.

```
Status XSendEvent(Display* display, Window w, Bool
↪  propagate, long event_mask, XEvent* event_send)
```

Sends a synthetic event to another client or window according to the destination window, propagation flag, and event mask.

- display Specifies the connection to the X server.
- *w* Specifies the window the event is to be sent to, or PointerWindow, or InputFocus.
- propagate Specifies a Boolean value.
- event_mask Specifies the event mask.
- event_send Specifies the event that is to be sent.
- **Possible errors**:
  - `BadValue`
  - `BadWindow`
- **Status return identifiers**:
  - `0`: Conversion to wire protocol format failed.
  - `Nonzero`: Conversion to wire protocol format succeeded.

```
unsigned long XDisplayMotionBufferSize(Display* display)
```

Returns the size of the motion history buffer for the specified display.

- **Possible errors**: None documented.

```
XTimeCoord* XGetMotionEvents(Display* display, Window w,
↪  Time start, Time stop, int* nevents_return)
```

Returns pointer motion history events for the specified window within the given time interval.

- display Specifies the connection to the X server.
- *w* Specifies the window.
- start, stop Specify the time interval in which the events are returned from the motion history buffer. You can pass a timestamp or CurrentTime.
- nevents_return Returns the number of events from the motion history buffer.
- **Possible errors**:
  - `BadWindow`

```
int (*XSetAfterFunction(Display* display, int
↪  (*procedure)()))()
```

Sets a procedure to be called after every Xlib protocol request, mainly for debugging or tracing request flow.

- display Specifies the connection to the X server.
- procedure Specifies the procedure to be called.
- **Possible errors**: None documented.

```
int (*XSynchronize(Display* display, Bool onoff))()
```

Enables or disables synchronous operation for the display connection, causing Xlib to wait for each request to complete when enabled.

- display Specifies the connection to the X server.
- onoff Specifies a Boolean value that indicates whether to enable or disable synchronization.
- **Possible errors**: None documented.

```
int (*XSetErrorHandler(handler))(Display*, XErrorEvent*)
    int (*handler)(Display *, XErrorEvent *)
```

Installs a custom handler for nonfatal X protocol errors and returns the previously installed handler.

- handler Specifies the program's supplied error handler.
- **Possible errors**: None documented.

```
XGetErrorText(Display* display, int code, char*
↪  buffer_return, int length)
```

Converts an X error code into a human-readable error message and stores it in the supplied buffer.

- display Specifies the connection to the X server.
- code Specifies the error code for which you want to obtain a description.
- buffer_return Returns the error description.
- length Specifies the size of the buffer.
- **Possible errors**: None documented.

```
XGetErrorDatabaseText(
    Display* display,
    char* name,
    char* message,
    char* default_string,
    char* buffer_return,
    int length
)
```

Looks up an error message string in the X error database and returns either the matching text or the supplied default string.

- display Specifies the connection to the X server.
- name Specifies the name of the application.
- message Specifies the type of the error message.
- default_string Specifies the default error message if none is found in the database.
- buffer_return Returns the error description.

– length Specifies the size of the buffer.

– **Possible errors**: None documented.

```
char* XDisplayName(char* string);
```

Returns the display name that Xlib would use for the supplied display string, or the default display name if the string is null.

– string Specifies the character string.

– **Possible errors**: None documented.

```
int (*XSetIOErrorHandler(handler))()
    int (*handler)(Display *);
```

Installs a custom handler for fatal I/O errors on a display connection and returns the previously installed handler.

– handler Specifies the program's supplied error handler.

– **Possible errors**: None documented.

## Input device functions

- **Structures**:

```c
/* Mask bits for ChangeKeyboardControl */
#define KBKeyClickPercent (1L<<0)
#define KBBellPercent (1L<<1)
#define KBBellPitch (1L<<2)
#define KBBellDuration (1L<<3)
#define KBLed (1L<<4)
#define KBLedMode (1L<<5)
#define KBKey (1L<<6)
#define KBAutoRepeatMode (1L<<7)

typedef struct {
    int key_click_percent;
    int bell_percent;
    int bell_pitch;
    int bell_duration;
    int led;
    int led_mode;           /* LedModeOn, LedModeOff */
    int key;
    int auto_repeat_mode;   /* AutoRepeatModeOff,
    ↪  AutoRepeatModeOn, AutoRepeatModeDefault */
} XKeyboardControl;
```

```c
typedef struct {
    int key_click_percent;
    int bell_percent;
    unsigned int bell_pitch, bell_duration;
    unsigned long led_mask;
    int global_auto_repeat;
    char auto_repeats[32];
} XKeyboardState;
```

- **Functions**:

```c
int XGrabPointer(
    Display* display,
    Window grab_window,
    Bool owner_events,
    unsigned int event_mask,
    int pointer_mode, keyboard_mode,
    Window confine_to,
    Cursor cursor,
    Time time
)
```

Actively grabs control of the pointer so pointer events are delivered according to the grab settings rather than normal event delivery rules.

– display Specifies the connection to the X server.

- grab_window Specifies the grab window.
- owner_events Specifies a Boolean value that indicates whether the pointer events are to be reported as usual or reported with respect to the grab window if selected by the event mask.
- event_mask Specifies which pointer events are reported to the client. The mask is the bitwise inclusive OR of the valid pointer event mask bits.
- pointer_mode Specifies further processing of pointer events. You can pass GrabModeSync or GrabModeAsync.
- keyboard_mode Specifies further processing of keyboard events. You can pass GrabModeSync or GrabModeAsync.
- confine_to Specifies the window to confine the pointer in or None.
- cursor Specifies the cursor that is to be displayed during the grab or None.
- time Specifies the time. You can pass either a timestamp or CurrentTime.

**Possible errors**:

- `BadCursor`
- `BadValue`
- `BadWindow`

**Possible Status identifiers**:

- `GrabSuccess`
- `AlreadyGrabbed`
- `GrabInvalidTime`
- `GrabNotViewable`
- `GrabFrozen`

```
XUngrabPointer(Display* display, Time time)
```

Releases an active pointer grab and restores normal pointer event processing.

- display Specifies the connection to the X server.
- time Specifies the time. You can pass either a timestamp or CurrentTime.

**Possible errors**:

- None.

```
XChangeActivePointerGrab(Display* display, unsigned int
↪  event_mask, Cursor cursor, Time time)
```

Changes the event mask and cursor associated with the current active pointer grab.

- display Specifies the connection to the X server.
- event_mask Specifies which pointer events are reported to the client. The mask is the bitwise inclusive OR of the valid pointer event mask bits.
- cursor Specifies the cursor that is to be displayed or None.

– time Specifies the time. You can pass either a timestamp or CurrentTime.

**Possible errors**:

– `BadCursor`
– `BadValue`

```
XGrabButton(
    Display* display,
    unsigned int button,
    unsigned int modifiers,
    Window grab_window,
    Bool owner_events,
    unsigned int event_mask,
    int pointer_mode, keyboard_mode,
    Window confine_to,
    Cursor cursor
)
```

Establishes a passive pointer grab that becomes active when the specified button and modifier combination is pressed.

– display Specifies the connection to the X server.
– button Specifies the pointer button that is to be grabbed or AnyButton.
– modifiers Specifies the set of keymasks or AnyModifier. The mask is the bitwise inclusive OR of the valid keymask bits.
– grab_window Specifies the grab window.
– owner_events Specifies a Boolean value that indicates whether the pointer events are to be reported as usual or reported with respect to the grab window if selected by the event mask.
– event_mask Specifies which pointer events are reported to the client. The mask is the bitwise inclusive OR of the valid pointer event mask bits.
– pointer_mode Specifies further processing of pointer events. You can pass GrabModeSync or GrabModeAsync.
– keyboard_mode Specifies further processing of keyboard events. You can pass GrabModeSync or GrabModeAsync.
– confine_to Specifies the window to confine the pointer in or None.
– cursor Specifies the cursor that is to be displayed or None.

**Possible errors**:

– `BadAccess`
– `BadCursor`
– `BadValue`
– `BadWindow`

```
XUngrabButton(
    Display* display,
    unsigned int button,
    unsigned int modifiers,
    Window grab_window
)
```

Removes a passive button grab for the specified button, modifier combination, and grab window. **Possible errors**:

- `BadValue`
- `BadWindow`

```
int XGrabKeyboard(
    Display* display,
    Window grab_window,
    Bool owner_events,
    int pointer_mode, keyboard_mode,
    Time time
)
```

Actively grabs control of the keyboard so keyboard events are delivered according to the grab settings.

- display Specifies the connection to the X server.
- grab_window Specifies the grab window.
- owner_events Specifies a Boolean value that indicates whether the keyboard events are to be reported as usual.
- pointer_mode Specifies further processing of pointer events. You can pass GrabModeSync or GrabModeAsync.
- keyboard_mode Specifies further processing of keyboard events. You can pass GrabModeSync or GrabModeAsync.
- time Specifies the time. You can pass either a timestamp or CurrentTime.

**Possible errors**:

- `BadValue`
- `BadWindow`

**Possible Status identifiers**:

- `GrabSuccess`
- `AlreadyGrabbed`
- `GrabInvalidTime`
- `GrabNotViewable`
- `GrabFrozen`

```
XUngrabKeyboard(Display* display, Time time)
```

Releases an active keyboard grab and restores normal keyboard event processing.

- display Specifies the connection to the X server.
- time Specifies the time. You can pass either a timestamp or CurrentTime.

**Possible errors**:

- None.

```
XGrabKey(
    Display* display,
    int keycode,
    unsigned int modifiers,
    Window grab_window,
    Bool owner_events,
    int pointer_mode, keyboard_mode
)
```

Establishes a passive keyboard grab that becomes active when the specified key and modifier combination is pressed.

- display Specifies the connection to the X server.
- keycode Specifies the KeyCode or AnyKey.
- modifiers Specifies the set of keymasks or AnyModifier. The mask is the bitwise inclusive OR of the valid keymask bits.
- grab_window Specifies the grab window.
- owner_events Specifies a Boolean value that indicates whether the keyboard events are to be reported as usual.
- pointer_mode Specifies further processing of pointer events. You can pass GrabModeSync or GrabModeAsync.
- keyboard_mode Specifies further processing of keyboard events. You can pass GrabModeSync or GrabModeAsync.

**Possible errors**:

- `BadAccess`
- `BadValue`
- `BadWindow`

```
XUngrabKey(Display* display, int keycode, unsigned int
↪  modifiers, Window grab_window)
```

Removes a passive keyboard grab for the specified key, modifier combination, and grab window.

- display Specifies the connection to the X server.
- keycode Specifies the KeyCode or AnyKey.
- modifiers Specifies the set of keymasks or AnyModifier. The mask is the bitwise inclusive OR of the valid keymask bits.
- grab_window Specifies the grab window.

**Possible errors**:

- `BadValue`
- `BadWindow`

```
XAllowEvents(Display* display, int event_mode, Time time)
```

Releases, synchronizes, or replays frozen pointer and keyboard events after a synchronous grab.

- display Specifies the connection to the X server.
- event_mode Specifies the event mode. You can pass AsyncPointer, SyncPointer, AsyncKeyboard, SyncKeyboard, ReplayPointer, ReplayKeyboard, AsyncBoth, or SyncBoth.
- time Specifies the time. You can pass either a timestamp or CurrentTime.

**Possible errors**:

- `BadValue`

```
XWarpPointer(
    Display* display,
    Window src_w,
    Window dest_w,
    int src_x, src_y,
    unsigned int src_width, src_height,
    int dest_x, dest_y
)
```

Moves the pointer to a new position, optionally only if it is currently inside a specified source rectangle.

- display Specifies the connection to the X server.
- src_w Specifies the source window or None.
- dest_w Specifies the destination window or None.
- src_x, src_y Specify the x and y coordinates of the source rectangle.
- src_width, src_height Specify a rectangle in the source window.
- dest_x, dest_y Specify the x and y coordinates within the destination window.

**Possible errors**:

- `BadWindow`

```
XSetInputFocus(Display* display, Window focus, int
↪ revert_to, Time time)
```

Changes the window that receives keyboard input focus.

- display Specifies the connection to the X server.
- focus Specifies the window, PointerRoot, or None.
- revert_to Specifies where the input focus reverts to if the window becomes not viewable.
- You can pass RevertToParent, RevertToPointerRoot, or RevertToNone.
- time Specifies the time. You can pass either a timestamp or CurrentTime.

**Possible errors**:

- `BadMatch`

- BadValue
- BadWindow

```
XGetInputFocus(Display* display, Window* focus_return, int*
↪ revert_to_return)
```

Returns the current input focus window and the current focus revert policy.

- display Specifies the connection to the X server.
- focus_return Returns the focus window, PointerRoot, or None.
- revert_to_return Returns the current focus state, such as RevertToParent, RevertToPointerRoot, or RevertToNone.

**Possible errors**:

- None.

```
XChangeKeyboardControl(Display* display, unsigned long
↪ value_mask, XKeyboardControl values)
```

Changes selected keyboard control settings such as key click, bell, LEDs, and auto-repeat behavior.

- display Specifies the connection to the X server.
- value_mask Specifies which controls to change. This mask is the bitwise inclusive OR of the valid control mask bits.
- values Specifies one value for each bit set to 1 in the mask.

**Possible errors**:

- BadMatch
- BadValue

```
XGetKeyboardControl(Display* display, XKeyboardState*
↪ values_return)
```

Retrieves the current keyboard control settings from the X server.

- display Specifies the connection to the X server.
- values_return Returns the current keyboard controls in the specified XKeyboardState structure.

**Possible errors**:

- None.

```
XAutoRepeatOn(Display* display)
```

Enables keyboard auto-repeat for the display.

- display Specifies the connection to the X server.

**Possible errors**:

- None.

```
XAutoRepeatOff(Display* display)
```

Disables keyboard auto-repeat for the display.

- display Specifies the connection to the X server.

**Possible errors**:

- None.

```
XBell(Display* display, int percent)
```

Rings the keyboard bell at a volume relative to the configured bell volume.

- display Specifies the connection to the X server.
- percent Specifies the volume for the bell, which can range from -100 to 100 inclusive.

**Possible errors**:

- `BadValue`

```
XQueryKeymap(Display* display, char keys_return[32])
```

Returns a bitmap describing which physical keys are currently pressed.

- display Specifies the connection to the X server.
- keys_return Returns an array of bytes that identifies which keys are pressed down. Each bit represents one key of the keyboard.

**Possible errors**:

- None.

```
int XSetPointerMapping(Display* display, unsigned char
↪  map[], int nmap)
```

Changes the pointer button mapping, which controls how physical buttons map to logical button numbers.

- display Specifies the connection to the X server.
- map Specifies the mapping list.
- nmap Specifies the number of items in the mapping list.

**Possible errors**:

- `BadValue`

**Possible Status identifiers**:

- `MappingSuccess`
- `MappingBusy`

```
int XGetPointerMapping(Display* display, unsigned char
↪  map_return[], int nmap)
```

Retrieves the current pointer button mapping.

- display Specifies the connection to the X server.
- map_return Returns the mapping list.
- nmap Specifies the number of items in the mapping list.

**Possible errors**:

- None.

```
XChangePointerControl(
    Display* display,
    Bool do_accel,
    Bool do_threshold,
    int accel_numerator,
    int accel_denominator,
    int threshold
)
```

Changes pointer acceleration and threshold settings for the display.

- display Specifies the connection to the X server.
- do_accel Specifies a Boolean value that controls whether the values for the accel_numerator or accel_denominator are used.
- do_threshold Specifies a Boolean value that controls whether the value for the threshold is used.
- accel_numerator Specifies the numerator for the acceleration multiplier.
- accel_denominator Specifies the denominator for the acceleration multiplier.
- threshold Specifies the acceleration threshold.

**Possible errors**:

– BadValue

```
XGetPointerControl(
    Display* display,
    int* accel_numerator_return,
    int* accel_denominator_return,
    int* threshold_return
)
```

Retrieves the current pointer acceleration numerator, denominator, and threshold settings.

– display Specifies the connection to the X server.

– accel_numerator_return Returns the numerator for the acceleration multiplier.

– accel_denominator_return Returns the denominator for the acceleration multiplier.

– threshold_return Returns the acceleration threshold.

**Possible errors**:

– None.

## Application utility functions

- **Functions**:

```
KeySym XLookupKeysym(XKeyEvent* key_event, int index)
```

The XLookupKeysym function returns the KeySym from a keyboard event by using the event's KeyCode and the specified KeySym index.

  - key_event Specifies the KeyPress or KeyRelease event.
  - index Specifies the index into the KeySyms list for the event's KeyCode.

**Possible errors**:

  - None.

```
KeySym XKeycodeToKeysym(Display* display, KeyCode keycode,
↪  int index)
```

The XKeycodeToKeysym function converts a KeyCode and KeySym index into the corresponding KeySym.

  - display Specifies the connection to the X server.
  - keycode Specifies the KeyCode.
  - index Specifies the element of KeyCode vector.

**Possible errors**:

  - None.

```
KeyCode XKeysymToKeycode(Display* display, KeySym keysym)
```

The XKeysymToKeycode function returns a KeyCode that is associated with the specified KeySym.

  - display Specifies the connection to the X server.
  - keysym Specifies the KeySym that is to be searched for.

**Possible errors**:

  - None.

```
XRefreshKeyboardMapping(XMappingEvent* event_map)
```

The XRefreshKeyboardMapping function updates Xlib's internal keyboard mapping information after a MappingNotify event.

  - event_map Specifies the mapping event that is to be used.

**Possible errors**:

– None.

```
void XConvertCase(KeySym keysym, KeySym* lower_return,
↪  KeySym* upper_return)
```

The XConvertCase function converts a KeySym into its lowercase and uppercase KeySym equivalents when such forms exist.

– keysym Specifies the KeySym that is to be converted.
– upper_return Returns the uppercase form of keysym, or keysym.
– lower_return Returns the lowercase form of keysym, or keysym.

**Possible errors**:

– None.

```
KeySym XStringToKeysym(char* string)
```

The XStringToKeysym function converts a textual KeySym name into its corresponding KeySym value.

– string Specifies the name of the KeySym that is to be converted.

**Possible errors**:

– None.

```
char *XKeysymToString(KeySym keysym)
```

The XKeysymToString function converts a KeySym value into its symbolic string name.

– keysym Specifies the KeySym that is to be converted.

**Possible errors**:

– None.

```
IsCursorKey(KeySym keysym)
```

The IsCursorKey macro tests whether the specified KeySym represents a cursor-control key, such as an arrow key.

– keysym Specifies the KeySym that is to be tested.

**Possible errors**:

– None.

```
IsFunctionKey(KeySym keysym)
```

The IsFunctionKey macro tests whether the specified KeySym represents a function key.

– keysym Specifies the KeySym that is to be tested.

**Possible errors**:

– None.

- `IsKeypadKey(KeySym keysym)`

The IsKeypadKey macro tests whether the specified KeySym represents a keypad key.

– keysym Specifies the KeySym that is to be tested.

**Possible errors**:

– None.

- `IsPrivateKeypadKey(KeySym keysym)`

The IsPrivateKeypadKey macro tests whether the specified KeySym is in the private keypad KeySym range.

– keysym Specifies the KeySym that is to be tested.

**Possible errors**:

– None.

- `IsMiscFunctionKey(KeySym keysym)`

The IsMiscFunctionKey macro tests whether the specified KeySym represents a miscellaneous function key.

– keysym Specifies the KeySym that is to be tested.

**Possible errors**:

– None.

- `IsModifierKey(KeySym keysym)`

The IsModifierKey macro tests whether the specified KeySym represents a keyboard modifier key.

– keysym Specifies the KeySym that is to be tested.

**Possible errors**:

– None.

- `IsPFKey(KeySym keysym)`

The IsPFKey macro tests whether the specified KeySym represents a programmable function key.

– keysym Specifies the KeySym that is to be tested.

**Possible errors**:

– None.

# ICCCM functions

- **Structures**:

```
typedef struct {
    unsigned char *value;       /* property data */
    Atom encoding;              /* type of property */
    int format;                 /* 8, 16, or 32 */
    unsigned long nitems;       /* number of items in value */
} XTextProperty;
```

```
* Window manager hints mask bits */
#define InputHint               (1L << 0)
#define StateHint               (1L << 1)
#define IconPixmapHint          (1L << 2)
#define IconWindowHint          (1L << 3)
#define IconPositionHint        (1L << 4)
#define IconMaskHint            (1L << 5)
#define WindowGroupHint         (1L << 6)
#define UrgencyHint             (1L << 8)
#define AllHints
    (InputHint|StateHint|IconPixmapHint|
                                IconWindowHint|
                                IconPositionHint
                                IconMaskHint|WindowGroupHint)
```

```
typedef struct {
    long flags;                 /* marks which fields in this structure are defined */
    Bool input;                 /* does this application rely on the window manager to get keyboard input? */
    int initial_state;          /* see below */
    Pixmap icon_pixmap;         /* pixmap to be used as icon */
    Window icon_window;         /* window to be used as icon */
    int icon_x, icon_y;         /* initial position of icon */
    Pixmap icon_mask;           /* pixmap to be used as mask for icon_pixmap */
    XID window_group;           /* id of related window group */
    /* this structure may be extended in the future */
} XWMHints;
```

```
#define WithdrawnState 0
#define NormalState 1           /* most applications start
    ↪ this way */
#define IconicState 3           /* application wants to start
    ↪ as an icon */
```

```
/* Size hints mask bits */
#define USPosition      (1L << 0)       /* user specified x,
    ↪ y */
#define USSize          (1L << 1)       /* user specified
    ↪ width, height */
#define PPosition       (1L << 2)       /* program specified
    ↪ position */
#define PSize           (1L << 3)       /* program specified
    ↪ size */
#define PMinSize        (1L << 4)       /* program specified
    ↪ minimum size */
#define PMaxSize        (1L << 5)       /* program specified
    ↪ maximum size */
#define PResizeInc      (1L << 6)       /* program specified
    ↪ resize increments */
#define PAspect         (1L << 7)       /* program specified
    ↪ min and max aspect ratios */
#define PBaseSize       (1L << 8)
#define PWinGravity     (1L << 9)
#define PAllHints   (PPosition|PSize| PMinSize|PMaxSize|
    ↪ PResizeInc|PAspect)
```

```
typedef struct {
    long flags;                     /* marks which fields in
        ↪ this structure are defined */
    int x, y;                       /* Obsolete */
    int width, height;              /* Obsolete */
    int min_width, min_height;
    int max_width, max_height;
    int width_inc, height_inc;

    struct {
        int x;                      /* numerator */
        int y;                      /* denominator */
    } min_aspect, max_aspect;

    int base_width, base_height;
    int win_gravity;
    /* this structure may be extended in the future */
} XSizeHints;
```

```
typedef struct {
    char *res_name;
    char *res_class;
} XClassHint;
```

```
typedef struct {
    int min_width, min_height;
    int max_width, max_height;
    int width_inc, height_inc;
} XIconSize;
```

- **Functions**:

```
Status XIconifyWindow(Display* display, Window w, int
    ↪ screen_number)
```

Requests that the window manager iconify the specified window.

  - display Specifies the connection to the X server.
  - $w$ Specifies the window.
  - screen_number Specifies the appropriate screen number on the host server.
  - **Possible errors**: None documented.
  - **Possible Status identifiers**: `True`, `False`.

```
Status XWithdrawWindow(Display* display, Window w, int
    ↪ screen_number)
```

Requests that the window manager withdraw the specified window from normal window management.

  - display Specifies the connection to the X server.
  - $w$ Specifies the window.
  - screen_number Specifies the appropriate screen number on the host server.
  - **Possible errors**: `BadWindow`.
  - **Possible Status identifiers**: `True`, `False`.

```
Status XReconfigureWMWindow(Display* display, Window w, int
    ↪ screen_number, unsigned int value_mask, XWindowChanges*
    ↪ values)
```

Reconfigures a top-level window through the window manager when the window is managed.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- screen_number Specifies the appropriate screen number on the host server.
- value_mask Specifies which values are to be set using information in the values structure.
- This mask is the bitwise inclusive OR of the valid configure window values bits.
- values Specifies the XWindowChanges structure.
- **Possible errors**: `BadValue`, `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
0 void XSetTextProperty(Display* display, Window w,
  ↪ XTextProperty* text_prop, Atom property)
```

Sets a text property on the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- text_prop Specifies the XTextProperty structure to be used.
- property Specifies the property name.
- **Possible errors**: `BadAlloc`, `BadAtom`, `BadValue`, `BadWindow`.

```
0 Status XGetTextProperty(Display* display, Window w,
  ↪ XTextProperty* text_prop_return, Atom property)
```

Retrieves a text property from the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- text_prop_return Returns the XTextProperty structure.
- property Specifies the property name.
- **Possible errors**: `BadAtom`, `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
0 void XSetWMName(Display* display, Window w, XTextProperty*
  ↪ text_prop)
```

Sets the window manager name property for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- text_prop Specifies the XTextProperty structure to be used.
- **Possible errors**: `BadAlloc`, `BadAtom`, `BadValue`, `BadWindow`.

```
Status XGetWMName(Display* display, Window w, XTextProperty*
↪ text_prop_return)
```

Retrieves the window manager name property from the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- text_prop_return Returns the XTextProperty structure.
- **Possible errors**: `BadAtom`, `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
XStoreName(Display* display, Window w, char* window_name)
```

Stores a simple string name for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- window_name Specifies the window name, which should be a null-terminated string.
- **Possible errors**: `BadAlloc`, `BadWindow`.

```
Status XFetchName(Display* display, Window w, char**
↪ window_name_return)
```

Fetches the simple string name associated with the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- window_name_return Returns the window name.
- **Possible errors**: `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
void XSetWMIconName(Display* display, Window w,
↪ XTextProperty* text_prop)
```

Sets the window manager icon name property for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- text_prop Specifies the XTextProperty structure to be used.
- **Possible errors**: `BadAlloc`, `BadAtom`, `BadValue`, `BadWindow`.

```
Status XGetWMIconName(Display* display, Window w,
↪  XTextProperty* text_prop_return)
```

Retrieves the window manager icon name property from the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- text_prop_return Returns the XTextProperty structure.
- **Possible errors**: `BadAtom`, `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
XSetIconName (Display* display, Window w, char* icon_name)
```

Sets a simple string icon name for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- icon_name Specifies the icon name, which should be a null-terminated string.
- **Possible errors**: `BadAlloc`, `BadWindow`.

```
Status XGetIconName(Display* display, Window w, char**
↪  icon_name_return)
```

Retrieves the simple string icon name for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- icon_name_return Returns the window's icon name, which is a null-terminated string.
- **Possible errors**: `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
XWMHints *XAllocWMHints()
```

Allocates and returns an XWMHints structure.

- **Possible errors**: None documented. Returns `NULL` if insufficient client memory is available.

```
XSetWMHints (Display* display, Window w, XWMHints* wmhints)
```

Sets the window manager hints property for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- wmhints Specifies the XWMHints structure to be used.
- **Possible errors**: `BadAlloc`, `BadWindow`.

```
XWMHints *XGetWMHints(display, w)
```

Retrieves the window manager hints property from the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- **Possible errors**: `BadWindow`.

```
XSizeHints *XAllocSizeHints()
```

Allocates and returns an XSizeHints structure.

- **Possible errors**: None documented. Returns `NULL` if insufficient client memory is available.

```
void XSetWMNormalHints(Display* display, Window w,
    XSizeHints* hints)
```

Sets the normal size hints for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- hints Specifies the size hints for the window in its normal state.
- **Possible errors**: `BadAlloc`, `BadWindow`.

```
Status XGetWMNormalHints(Display* display, Window w,
    XSizeHints* hints_return, long* supplied_return)
```

Retrieves the normal size hints for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- hints_return Returns the size hints for the window in its normal state.
- supplied_return Returns the hints that were supplied by the user.
- **Possible errors**: `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
void XSetWMSizeHints(Display* display, Window w, XSizeHints*
↪  hints, Atom property)
```

Sets size hints on the specified window using the given property.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- hints Specifies the XSizeHints structure to be used.
- property Specifies the property name.
- **Possible errors**: `BadAlloc`, `BadAtom`, `BadWindow`.

```
Status XGetWMSizeHints(
    Display* display,
    Window w,
    XSizeHints* hints_return,
    long* supplied_return,
    Atom property
)
```

Retrieves size hints from the specified window using the given property.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- hints_return Returns the XSizeHints structure.
- supplied_return Returns the hints that were supplied by the user.
- property Specifies the property name.
- **Possible errors**: `BadAtom`, `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
XClassHint *XAllocClassHint()
```

Allocates and returns an XClassHint structure.

- **Possible errors**: None documented. Returns `NULL` if insufficient client memory is available.

```
XSetClassHint(Display* display, Window w, XClassHint*
↪ class_hints)
```

Sets the resource class and resource name hints for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- class_hints Specifies the XClassHint structure that is to be used.
- **Possible errors**: `BadAlloc`, `BadWindow`.

```
Status XGetClassHint(Display* display, Window w, XClassHint*
↪ class_hints_return)
```

Retrieves the resource class and resource name hints for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- class_hints_return Returns the XClassHint structure.
- **Possible errors**: `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
XSetTransientForHint (Display* display, Window w, Window
↪ prop_window)
```

Sets the WM_TRANSIENT_FOR property for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- prop_window Specifies the window that the WM_TRANSIENT_FOR property is to be set to.
- **Possible errors**: `BadAlloc`, `BadWindow`.

```
Status XGetTransientForHint(Display* display, Window w,
↪ Window* prop\_window\_return)
```

Retrieves the WM_TRANSIENT_FOR property for the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- prop_window_return Returns the WM_TRANSIENT_FOR property of the specified window.
- **Possible errors**: `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
Status XSetWMProtocols(Display* display, Window w, Atom*
↪  protocols, int count)
```

Sets the list of window manager protocols supported by the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- protocols Specifies the list of protocols.
- count Specifies the number of protocols in the list.
- **Possible errors**: `BadAlloc`, `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
Status XGetWMProtocols(Display* display, Window w, Atom**
↪  protocols_return, int* count_return)
```

Retrieves the list of window manager protocols supported by the specified window.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- protocols_return Returns the list of protocols.
- count_return Returns the number of protocols in the list.
- **Possible errors**: `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
Status XSetWMColormapWindows(Display* display, Window w,
↪  Window* colormap_windows, int count)
```

Sets the list of windows whose colormaps the window manager should install.

- display Specifies the connection to the X server.
- $w$ Specifies the window.
- colormap_windows Specifies the list of windows.
- count Specifies the number of windows in the list.
- **Possible errors**: `BadAlloc`, `BadWindow`.

- **Possible Status identifiers**: `True`, `False`.

```
Status XGetWMColormapWindows(Display* display, Window w,
    ↪ Window** colormap_windows_return, int count_return)
```

Retrieves the list of windows whose colormaps are associated with the specified window.

- display Specifies the connection to the X server.
- w Specifies the window.
- colormap_windows_return Returns the list of windows.
- count_return Returns the number of windows in the list.
- **Possible errors**: `BadWindow`.
- **Possible Status identifiers**: `True`, `False`.

```
XIconSize *XAllocIconSize( )
```

Allocates and returns an XIconSize structure.

- **Possible errors**: None documented. Returns `NULL` if insufficient client memory is available.

```
XSetIconSizes(Display* display, Window w, XIconSize*
    ↪ size_list, int count)
```

Sets the icon size list for the specified window.

- display Specifies the connection to the X server.
- w Specifies the window.
- size_list Specifies the size list.
- count Specifies the number of items in the size list.
- **Possible errors**: `BadAlloc`, `BadWindow`.

```
Status XGetIconSizes(Display* display, Window w, XIconSize**
    ↪ size_list_return, int* count_return)
```

Retrieves the icon size list for the specified window.

- display Specifies the connection to the X server.
- w Specifies the window.
- size_list_return Returns the size list.
- count_return Returns the number of items in the size list.
- **Possible errors**: `BadWindow`.

– **Possible Status identifiers**: `True`, `False`.

```
void XmbSetWMProperties(
    Display* display,
    Window w,
    char* window_name, icon_name, argv[],
    int argc,
    XSizeHints* normal_hints,
    XWMHints* wm_hints,
    XClassHint* class_hints
)
```

Sets several standard window manager properties using multibyte string arguments.

– display Specifies the connection to the X server.
– w Specifies the window.
– window_name Specifies the window name, which should be a null-terminated string.
– icon_name Specifies the icon name, which should be a null-terminated string.
– argv Specifies the application's argument list.
– argc Specifies the number of arguments.
– hints Specifies the size hints for the window in its normal state.
– wm_hints Specifies the XWMHints structure to be used.
– class_hints Specifies the XClassHint structure to be used.

– **Possible errors**: `BadAlloc`, `BadWindow`.

```
void XSetWMProperties(
    Display* display,
    Window w,
    XTextProperty* window_name, icon_name,
    char** argv,
    int argc,
    XSizeHints* normal_hints,
    XWMHints* wm_hints,
    XClassHint* class_hints
)
```

Sets several standard window manager properties for the specified window.

– display Specifies the connection to the X server.
– w Specifies the window.
– window_name Specifies the window name, which should be a null-terminated string.
– icon_name Specifies the icon name, which should be a null-terminated string.
– argv Specifies the application's argument list.
– argc Specifies the number of arguments.

– normal_hints Specifies the size hints for the window in its normal state.
– wm_hints Specifies the XWMHints structure to be used.
– class_hints Specifies the XClassHint structure to be used.
– **Possible errors**: `BadAlloc`, `BadWindow`.

## Errors and error functions

- **Error defines**:

```
#define Success             0
#define BadRequest          1
#define BadValue            2
#define BadWindow           3
#define BadPixmap           4
#define BadAtom             5
#define BadCursor           6
#define BadFont             7
#define BadMatch            8
#define BadDrawable         9
#define BadAccess           10
#define BadAlloc            11
#define BadColor            12
#define BadGC               13
#define BadIDChoice         14
#define BadName             15
#define BadLength           16
#define BadImplementation   17

#define LASTError           18
```

- **Structures**:

```
typedef struct {
    int type;
    Display *display;                   /* Display the event
    ↪  was read from */
    unsigned long serial;               /* serial number of
    ↪  failed request */
    unsigned char error_code;           /* error code of
    ↪  failed request */
    unsigned char request_code;         /* Major op-code of
    ↪  failed request */
    unsigned char minor_code;           /* Minor op-code of
    ↪  failed request */
    XID resourceid;                     /* resource id */
} XErrorEvent;
```

- **Functions**:

```
int (*XSetErrorHandler(handler))()
    int (*handler)(Display *, XErrorEvent *)
```

The `XSetErrorHandler` function sets the handler that Xlib calls when an X protocol error occurs. It returns the previously installed error handler.

– handler Specifies the program's supplied error handler.

- **Possible errors**:
  - None.

```
XGetErrorText(Display* display, int code, char*
↪  buffer_return, int length)
```

The `XGetErrorText` function obtains a textual description for the specified X error code and stores it in the supplied buffer.

- display Specifies the connection to the X server.
- code Specifies the error code for which you want to obtain a description.
- buffer_return Returns the error description.
- length Specifies the size of the buffer.

- **Possible errors**:
  - None.

```
XGetErrorDatabaseText(
    Display* display,
    char* name,
    char* fmessage,
    char* default_string,
    char* buffer_return,
    int length
)
```

The `XGetErrorDatabaseText` function looks up an error message in the X error database and returns either the matching message or the supplied default string.

- display Specifies the connection to the X server.
- name Specifies the name of the application.
- fmessage Specifies the type of the error message.
- default_string Specifies the default error message if none is found in the database.
- buffer_return Returns the error description.
- length Specifies the size of the buffer.

- **Possible errors**:
  - None.

```
char* XDisplayName(char* string);
```

The `XDisplayName` function returns the display name that Xlib would use for the specified display string.

- string Specifies the character string.

- **Possible errors**:
  - None.

```
int (*XSetIOErrorHandler(handler))()
    int (*handler)(Display *);
```

The `XSetIOErrorHandler` function sets the handler that Xlib calls when a fatal I/O error occurs on a display connection.

- handler Specifies the program's supplied error handler.
- **Possible errors**:
  - None.

## Memory management functions

- **Functions**:

```
XFree(void* data)
```

Frees memory allocated by Xlib functions, such as returned strings, arrays, or structures.

- **Possible errors**: None.

```
XFreeColormap(Display *display, Colormap colormap)
```

Destroys the specified colormap and frees the server-side resources associated with it.

- **display**: Specifies the connection to the X server.
- **colormap**: Specifies the colormap that you want to destroy.
- **Possible errors**: BadColor.

```
int XFreeColors(
    Display *display,
    Colormap colormap,
    unsigned long pixels[],
    int npixels,
    unsigned long planes
)
```

Frees the specified color cells in a colormap so that they may be reused.

- *display*: Specifies the connection to the X server.
- *colormap*: Specifies the colormap.
- *pixels*: Specifies an array of pixel values that map to the cells in the specified colormap.
- *npixels*: Specifies the number of pixels.
- *planes*: Specifies the planes you want to free.
- **Possible errors**: BadAccess, BadColor, BadValue.

```
XFreeFontPath(char** list)
```

Frees the font path list returned by XGetFontPath.

- list Specifies the array of strings you want to free.
- **Possible errors**: None.

```
XFreePixmap(Display* display, Pixmap pixmap)
```

Frees the specified pixmap and releases its associated server-side storage.

- display Specifies the connection to the X server.
- pixmap Specifies the pixmap.
- **Possible errors**: BadPixmap.

```
XFreeCursor(Display* display, Cursor cursor)
```

Frees the specified cursor when it is no longer needed by the client.

- display Specifies the connection to the X server.
- cursor Specifies the cursor.
- **Possible errors**: BadCursor.